expert sleepers disting NT

User manual

Version 1.15

And you may ask yourself, "How do I work this?"

- *Talking Heads, “Once in a Lifetime”*

Copyright © 2026 Expert Sleepers Ltd. All rights reserved.

This manual, as well as the hardware and software described in it, is furnished under licence and may be used or copied only in accordance with the terms of such licence. The content of this manual is furnished for informational use only, is subject to change without notice, and should not be construed as a commitment by Expert Sleepers Ltd. Expert Sleepers Ltd assumes no responsibility or liability for any errors or inaccuracies that may appear in this document.

Expert Sleepers® is a registered trade mark in the UK, the European Union, and the United States.

Table of Contents

Introduction....8
Preamble....8
What does it do?....9
What is it not?....9
Getting started....10
Example presets....10
Installation....10
Controls....15
Navigation....16
Inputs and outputs....20
USB....21
MIDI connections....21
MicroSD card....22
Signal flow....29
Algorithm specifications....30
Sample rate and buffer size....31
CPU usage/overload....31
Preset editor/routing analyser....33
NT Helper....34
Performance page....35
Menu reference....38
Algorithms menu....39
Presets menu....40
Mappings menu....43
Settings menu....47
Misc(ellaneous) menu....52
UI Scripts menu....55
Algorithm-specific menu....55
Common algorithm features....56
Common polysynth features....58
Plug-ins....65

Algorithm reference....68
Accent sweep....69
Arpeggiator....71
Attenuverter....77
Audio recorder....78
Augustus Loop....80
Auto-calibrator....87
Auto-sampler....90
Bit Crusher....93
Chaos....95
Chorus/Flange....97
Chorus (Vintage)....99
Clock....100
Clock Divider....104
Clock Multiplier....106
Compressor....107
Convolver....109
Crossfader....112
Debouncer....114
Delay (Mono)....115
Delay (Stereo)....117
Delay (Tape)....119
Delayed Function....121
DJ Filter....124
Dream Machine....126
EQ Parametric....130
Envelope (AR/AD)....132
Envelope (DAHDSR)....135
Envelope Follower....139
Envelope Sequencer....140
ES-5 Encoder....144
ESX-8CV Combiner....145
Euclidean Patterns....147

Feedback Receive/Send....150
Filter bank....152
Frequency Shifter....155
Granulator....157
Kirbinator....166
LFO....171
Linear/Exponential....174
Logic....176
Looper....179
Looper (MicroSD)....190
Lua Expression....192
Lua Script....194
Macro Oscillator....196
Macro Oscillator 2....199
MIDI CC Generator....202
MIDI Filter....204
MIDI Player....205
MIDI Triggers....209
Minimum/maximum....211
Mixer Mono....212
Mixer Stereo....214
Multi-Switch....216
Noise gate....219
Noise generator....221
Notes....222
Oscilloscope....223
Phase Shifter....225
Phaser....227
Pitch reference....229
Pitch Shifter....231
Poly CV....233
Poly FM....237
Poly FM (Editable)....240

Poly Macro Oscillator 2....242
Poly Multisample....246
Poly Multisample (legacy)....251
Poly Resonator....254
Poly Wavetable....258
Quadraphonic Mixer....264
Quantizer....268
Rectifier....274
Resonator....275
Reverb....278
Reverb (Clouds)....280
Sample and Hold....282
Sample Player....283
Sample Players....286
Sample Player (Clocked)....292
Sample Player (Scrub)....295
Sample Player (Sliced)....297
Sample Player (Speed)....301
Saturation....302
Seaside Jawari....303
Shift Register Random....306
Slew rate limiter....309
Slope Detector....311
Spectral Conquest....313
Spectral Freeze....318
Spectral Vocoder....322
Spectrum Analyzer....324
SSB Modulator....325
Step Sequencer....327
Step Sequencer Head....334
Stopwatch....337
Temporal Freeze....339
Temporal MIDI Quantizer....342

Texture Synthesizer....344
Three Pot....346
Tracker....350
Transient Detector....354
Tuner (fancy)....355
Tuner (simple)....357
USB audio (from host)....358
USB audio (to host)....359
VCA/Multiplier....360
VCF (State Variable)....361
VCO - Pulsar....364
VCO - Waveshaping....367
VCO - Wavetable....370
Vocoder....372
Waveform Animator....374
Waveshaper....376
Wavetable Waveshaper....378
UI Scripts....380
MIDI SysEx reference....381
I2C reference....392
Updating the firmware....395
Acknowledgments....401

Introduction

Congratulations on your purchase of an Expert Sleepers disting NT. Please read this user manual before operating your new module.

A note on navigating this manual

When one part of the manual refers to another, it may say something like “see Settings, below”. In such cases the word 'below' (or 'above') is a hyperlink, and can be clicked on. Try it.

It seems very technical

This is intended as a reference manual, not a tutorial. Please visit our YouTube channel for some friendlier demos, or just ask for help.

Where to get help

Email, forum, and social media links can be found at the bottom of every page on our website[1]. We also have a Discord server[2].

We prefer support requests to be made through the forum, if possible – failing that, by email.

If you’re lucky enough to have a local bricks and mortar modular store, please visit them, try things out, and get help there. And most of all, buy your modules from them too.

Preamble

The disting NT is the latest in the line of modules from Expert Sleepers that started with the original disting in 2014, which performed one of 16 very different functions at the turn of a knob. This concept has evolved through the disting mk3 (2015) – which added the possibility of firmware updates, and so a continually improving selection of functions – and the disting mk4 (2017) – which added a display, and brought the MicroSD card slot onto the front of the module, emphasising its increased importance for functions such as sample playback – to the disting NT’s predecessor, the disting EX (2020). The EX (finally) added an OLED display, and a much more powerful CPU platform, taking the functions it could perform to the next level.

Ever since the mk3 was released, the modules’ firmwares have been continuously updated with new features, making them even more useful and better value.[3]

We aim to continue this legacy with the disting NT.

Whereas the disting EX built directly on the selection of functions in the mk4, and then added some of

1 https://www.expert-sleepers.co.uk/

2 https://modwiggler.com/forum/viewtopic.php?t=287914

3 Some stats – the mk3 had 10 major updates in 2 years; the mk4 has (so far) had 64 major updates in 7 years; the EX has (so far) had 25 major updates in 4 years.

its own, with the disting NT we've taken everything back to square one. We anticipate (eventually) adding every function ever offered by the previous distings, but every algorithm has been reconsidered and reappraised in the light of the possibilities offered by the new hardware platform. Often things have been streamlined and simplified – for example, a disting EX algorithm that offered a polysynth, a chorus, and a delay would be implemented on the NT as three separate functions, which you could combine as you choose – each function therefore being smaller and easier to deal with.

While the exact future of this module is yet to be written, one thing is certain – it will be driven by user feedback. We have an active user community on the ModWiggler forum[4], and suggestions (and we'll admit, bug reports) left there have often made it into the firmware. So we strongly encourage you to get in touch! Let us know what the disting NT can be for you.

What does it do?

The module takes audio and/or CV input, runs a bunch of software processing, and generates audio and/or CV output. It does this continuously.

The chunks of software processing are called *algorithms*. You can think of them much as you would think of plug-ins in your DAW. The module has a number of algorithms to choose from, and you can add the ones you want in a big list and the module will run them all.

A collection of algorithms, their various parameters, and associated data such as MIDI mapping, together make up a *preset*.

Like plug-ins in a DAW, we use 'algorithm' to mean both the abstract *type* of processing you want to run, and a particular *instance* of that algorithm in a preset. For example, you might say "I'll load a reverb plug-in on this track" meaning the *type* of plug-in you want to load, and also "I changed the decay time on the reverb plug-in" to refer to the particular *instance* of the reverb plug-in.

What is it not?

The disting NT is not a "virtual modular". There are no virtual cables flying around. It simply collects together a variety of functions that are useful in the context of a modular synthesizer and makes it possible to use a number of them at once.

It is not a "patch player". It is not the intention that you do all the complex setup on a computer and then copy it over to the module. The module is designed to be used *in a modular*, with a UI that makes it fluid to do everything in the module itself.

It is not only useful via MIDI. The large number of inputs (and outputs) makes it possible to use it entirely via CV, interfacing with the rest of your modules in a natural way. (That said, it does also make a really good MIDI polysynth.)

4 https://www.modwiggler.com/forum/viewforum.php?f=35

Getting started

First, carefully read the installation instructions below.

Then, at least skim the ‘Navigation’ section.

This will enable you to load some of the example presets from the MicroSD card.

You might like to watch this quickstart video[5] before going much further, which was made to help get new users up and running.

Please also check if you have the latest firmware, which can be found on our website here[6].

Example presets

Some example presets are included on the MicroSD card supplied with the module, but by the time you read this there will almost certainly be more available on our GitHub[7]. Please look there for the latest examples, and for documentation of how the presets are put together, which can be quite informative.

Installation

House the module in a Eurorack case of your choosing. The power connector is 16-pin Doepfer standard[8]. If using the power cable supplied with the module, the red edge of the cable is furthest from the top edge of the PCB, and carries -12V. ("-12V" is marked on the PCB itself next to this end of the connector.) Be sure to connect the other end of the power cable correctly, again so -12V corresponds to the red stripe on the cable.



Please observe ESD (electro-static discharge) precautions when handling the module (and indeed, any module).

5 https://www.youtube.com/watch?v=EfkrNHkTKF0

6 https://expert-sleepers.co.uk/distingNTfirmwareupdates.html

7 https://github.com/expertsleepersltd/distingNT/tree/main/presets

8 http://www.doepfer.de/a100_man/a100t_e.htm

Please do not operate the module without it being securely fastened in a case.

Physical dimensions

The disting NT is 22HP wide and 25mm deep (including the depth added by the included power cable).

Power requirements

The disting NT draws 211mA on the +12V rail, and 96mA on the -12V rail, when idling. The actual power consumption is heavily dependent on how many input and output socket LEDs are lit. Each fully illuminated socket draws about 4mA (from the +12V rail or -12V rail depending on the polarity), so you could in theory pull another 72mA, but that is highly unlikely to occur in actual use – you'd have to somehow feed a steady 10V into every input socket and be running algorithms that were outputting a steady 10V from every output. When budgeting for case power it's probably safe to split that 72mA between the two rails and say it draws 247mA on the +12V rail and 132mA on the -12V rail. But really, if that 36mA is enough to put your PSU over the edge, you need a bigger PSU anyway.

The disting NT does not use the 5V rail.

Connecting expansion modules

Turn off the power before connecting or disconnecting expansion modules.

ES-5

Connect an ES-5[9] module via the header on the back of the disting NT marked J11 ES-5, using the 10-way cable provided with the ES-5. The red stripe should be oriented down on both modules, as shown in the photo below, and in the ES-5 user manual[10]. At the ES-5 end, connect the cable to the header marked "GT7/To ES-3".



The disting NT can use an attached ES-5 to output a considerable number of extra CVs and gates,

9 https://www.expert-sleepers.co.uk/es5.html

10 https://expert-sleepers.co.uk/es5usermanual.html

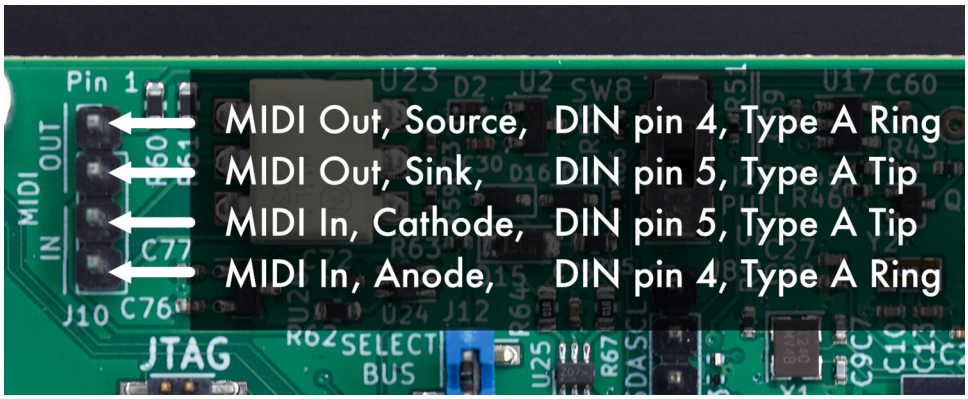
from the ES-5 itself and from attached ESX-8GT[11] and ESX-8CV[12] expanders. Some algorithms may support direct use of the expanders, but in general any signals can be passed to the expanders via the ES-5 Encoder and ESX-8CV Combiner algorithms (below).

MIDI

Connect a MIDI breakout to the header on the back of the disting NT marked J10 MIDI.

MIDI can be used to remotely control the algorithm parameters, to share clocks, and to play notes in the synthesizer algorithms etc. The disting NT can also send MIDI to other devices, for example to sequence them, or simply to provide parameter feedback to a MIDI controller.

The image below shows how the header is wired, including pin numbers for 5-pin DIN connections, and tip/ring indications for a Type A TRS MIDI jack.



See below for more details on hooking up MIDI using the included breakout module.

I2C

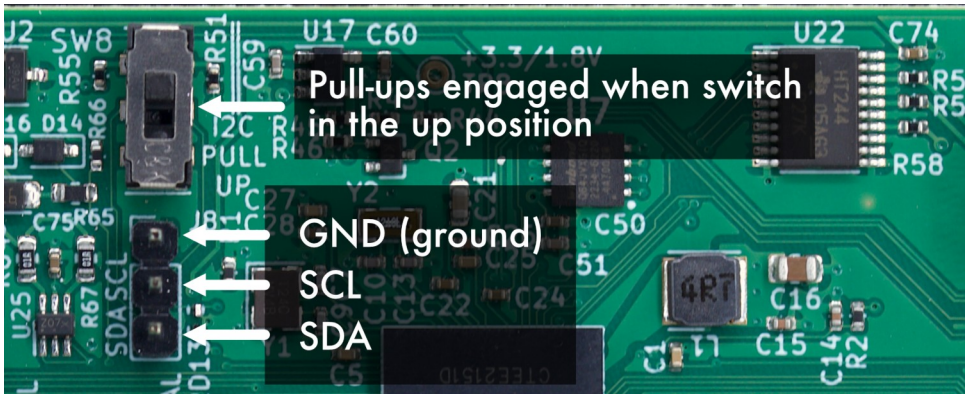
A device or module that communicates via I2C can be connected via the header marked J8. The SDA and SCL pins are marked; the third pin is ground.

Like MIDI, I2C can be used for remotely controlling algorithm parameters and playing notes. While less common than MIDI, it is the control mechanism of choice for the whole Teletype[13] module ecosystem.

11 https://expert-sleepers.co.uk/esx8gt.html

12 https://expert-sleepers.co.uk/esx8cv.html

13 https://monome.org/docs/teletype/



CVM-8

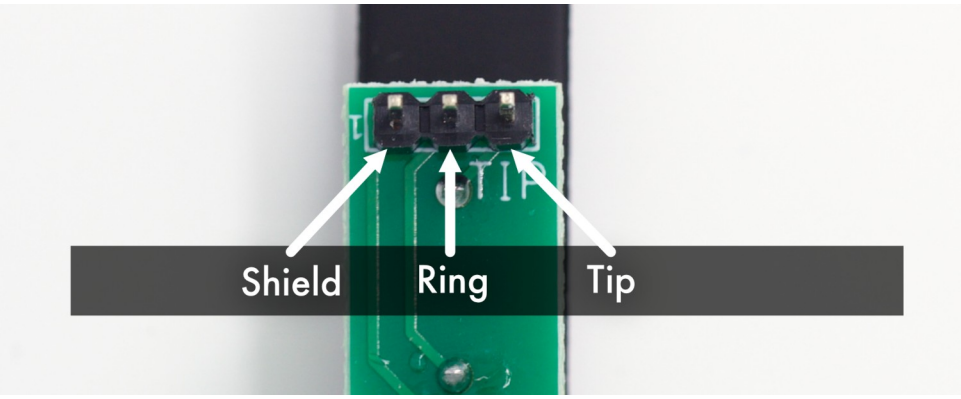
If you feel the need for more CV inputs on the disting NT, please take a look at the CVM-8[14], which has eight inputs and can connect via MIDI, I2C, or the Select Bus.

Using the included breakout module

The disting NT is supplied with a “Tiny MIDI Breakout” (aka TMB), a pre-existing Expert Sleepers product, which is simply six TRS jack sockets on a 2HP panel. The product page for the TMB is here[15]. In the box are also some jump cables (aka “DuPont cables” or “Arduino cables”) which you can use to connect the two.

If you need more, or longer, cables they are easily found online.

Each of the six TRS sockets is wired to a 3-pin header on the back of the PCB, like so:



Use the jump cables to connect the relevant pins of the TRS sockets to the pins of the breakout headers on the disting NT that you would like to use.

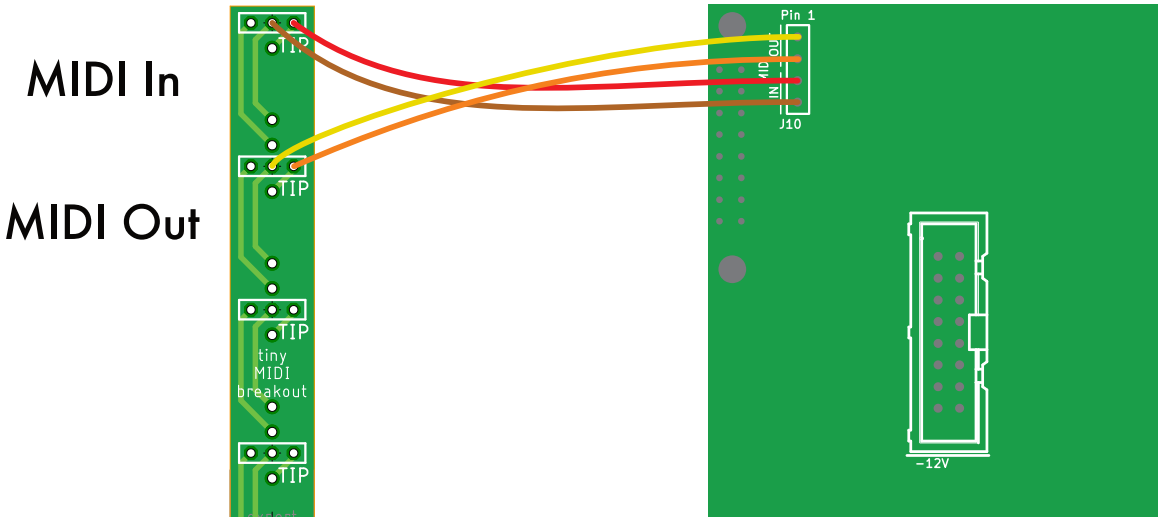
For example, to use one of the sockets as a TRS MIDI input, use two cables to connect the Tip and Ring of the socket to the MIDI In Tip & Ring as shown above.

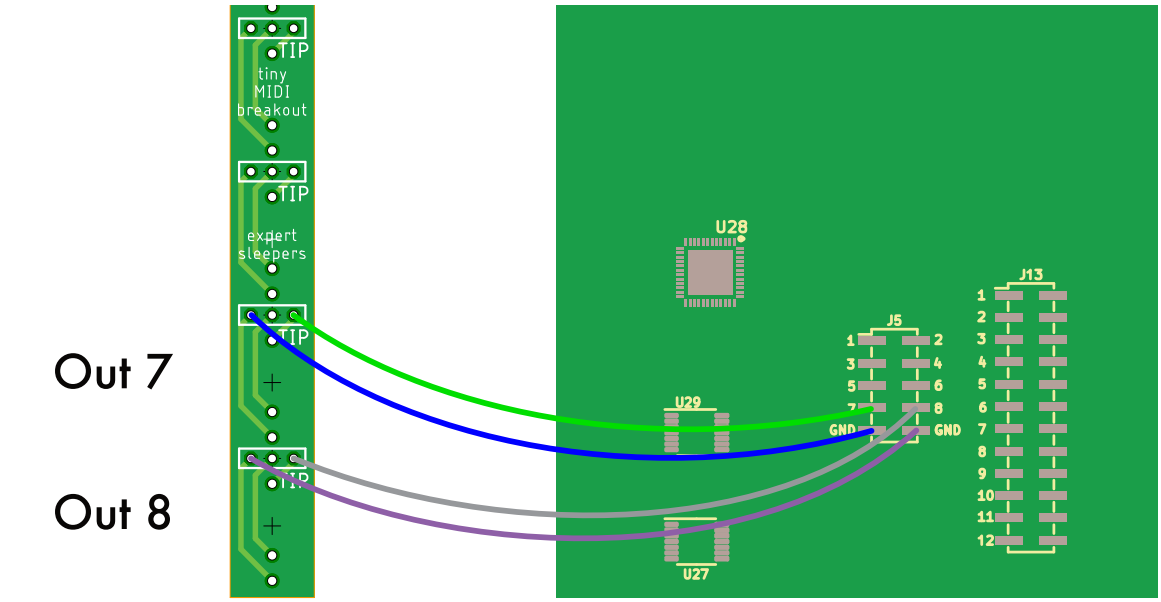
14 https://expert-sleepers.co.uk/cvm8.html

15 https://expert-sleepers.co.uk/tinymidibreakout.html

There is a video showing how to connect the breakout here[16].

The diagrams below shows typical connections when using the breakout for MIDI and audio. The wire colours match those shown in the video.



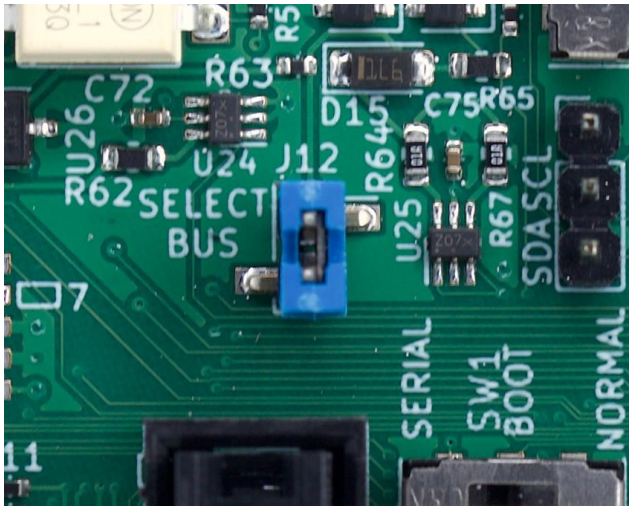


Jumpers/switches

The jumper on the board labelled J12 SELECT BUS connects the module's Select Bus circuitry to the power bus's CV line. Fit the jumper if you want to use the Select Bus; remove it if you use the CV

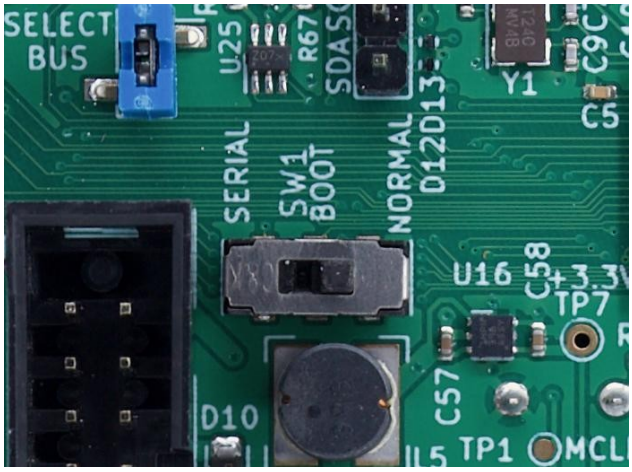
16 https://www.youtube.com/watch?v=NxiFBvlkGgo

bus for its originally intended purpose.



Use the switch marked SW8 I2C PULL UP to enable or disable the I2C pull-up resistors (image above). The pull-ups are enabled when the switch is in the ‘up’ position (nearest the top of the PCB). At least one device on the I2C bus needs to have pull-up resistors enabled. It may work fine with more than one set of pull-ups enabled, or it may not.

The switch marked SW1 BOOT can be used to force the module into serial bootloader mode. Normally you would leave the switch in the “Normal” position (to the right) and use the module’s menu to enter bootloader mode. If however this is not possible, put the switch into the “Serial” position (to the left) to force it into bootloader mode. Note though that you will then have to return it to “Normal” to boot normally after installing new firmware. See below for a description of the usual firmware update procedure.

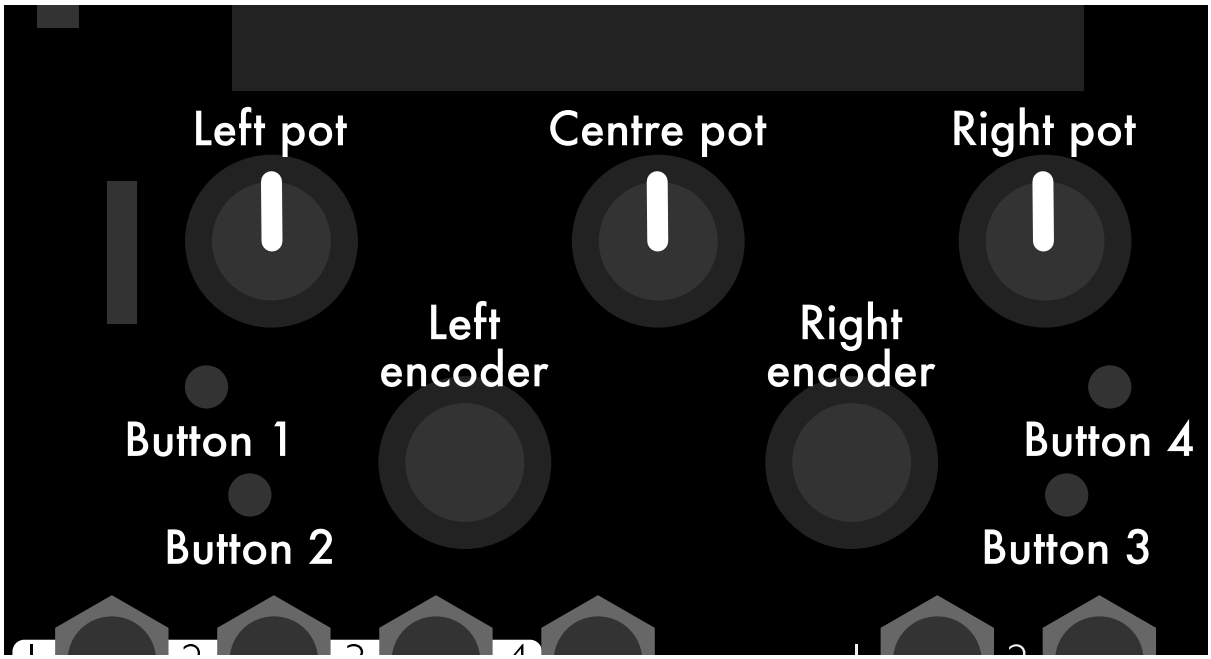


Controls

The disting NT has three pots, two encoders, and four pushbuttons. The pots and encoders also have a pushbutton action.

The controls are unlabelled, on the basis that their function can be and is completely redefined in software. Popup help can be displayed on the display to help you keep track.

In this manual, we will simply refer to “the left pot”, “the centre pot”, and “the right pot”; similarly, “the left encoder” and “the right encoder”. The pushbuttons are referred to as buttons 1-4, ordered from left to right.



There is a setting you can change to order the buttons in the opposite direction i.e. from right to left. See below.

Navigation

There are two states that the module UI will be in most of the time: the algorithm view, in which you navigate the various active algorithms and interact with their parameters; or the menu system.

Pop-up help is available, indicating the function of each control in a small area at the bottom of the screen. If you’ve disabled it in the Settings, you can still view the help at any time by holding down button 3.



In the pop-up help, words at the extreme left and right sides refer to the pushbuttons. Other help is, as far as space will allow, lined up above the encoder or pot it refers to. In the image above, ‘Menu’, ‘Params’, ‘Help’, and ‘Shift’ refer to pushbuttons 1-4 respectively. ‘Turn:Select’ and ‘Push:Algorithm’ both refer to the left encoder.

Menus

You can easily tell when you're in the menu system because there will be a dotted line border around the screen.



Button 1 enters the menu system; it also immediately exits the menu system if pressed while you're anywhere in the menus.

The left encoder navigates the menu: turn it to select a submenu or menu item, and press it to descend one level. Button 2 is 'back' i.e. it goes up one menu level. If you keep pressing button 2 you will eventually leave the menu entirely.

Where a menu allows you to change a value, or select from a list of items, you can in general do this either by turning the right encoder, or by pressing the left encoder to descend one more menu level and then turning the left encoder.

There is a setting to swap the function of the two encoders, if you find it preferable to primarily drive the menu with the right encoder. See below.

File browser

Some menus, for example the menu for loading presets, require you to browse for a file on the MicroSD card.

In the file browser, use the left encoder to move up and down the list of files, to select items, and to enter folders. Press the right encoder to leave a folder and go up one level of the folder hierarchy.

Editing text

Some menus, for example the menu to set the preset name, allow you to edit text.

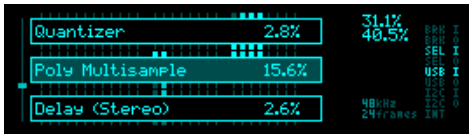
- Turn the left encoder to move the cursor, and the right encoder to change the character at the cursor position.
- Pressing the right encoder advances to the next character in the list ' ', '0', 'A', 'a'.
- Pressing the right pot inserts a space at the cursor position and moves following characters to the right.
- Pressing the left pot deletes the character at the cursor position and moves following characters to the left.
- Press the left encoder to accept the text and return to the menu above.

Algorithms view

This is where you'll spend most of your time interacting with the module. There are two primary display modes – the algorithms overview, where you can see all the running algorithms and the signals flowing between them, and the single algorithm view, where you edit the parameters of an algorithm.

Algorithms overview

Here you can see all the algorithms running, in a list starting with the module inputs at the top, and flowing down to the module outputs at the bottom.



Turning the left encoder or the left pot scrolls through the algorithm list.

The 'current algorithm' is the one highlighted, in the centre of the screen. This is the algorithm that will be the target of any menu commands e.g. Move up/down. It is also the algorithm that will be live for editing when you switch to the single algorithm view.

There is a 'scroll bar' graphic on the left side of the screen showing where you are in the list.

Holding button 4 and turning the left encoder moves the current algorithm up and down in the list, exactly as if you'd used the 'Move algorithm up/down' menu.

Holding button 4 and pressing the right encoder toggles the Bypass parameter of the current algorithm.

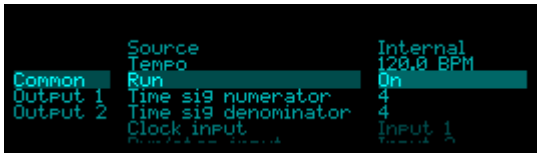
Press the right encoder to switch to a different view showing the input and output signals of the current algorithm. Turning the encoder or pots still scrolls through the list as before.

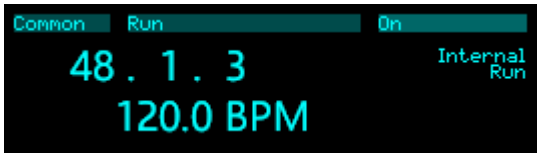


Press the left encoder or left pot to switch to the single algorithm view.

Single algorithm view

This is where you edit the parameters of an algorithm. There are two modes for this view: one which shows only parameters, arranged in pages, and one which shows a single parameter and devotes the rest of the screen to a display specific to the algorithm in question.





In either mode, navigation is exactly the same.

Turn the left pot to select the page of parameters to view.

Turn the centre pot or the left encoder to select a parameter within the current page.

Turn the right pot or the right encoder to change the parameter value.

The pot uses a form of ‘soft takeover’ rather than jumping to the value that corresponds directly to the knob position. When a new parameter is selected for editing, turning the pot clockwise will always increase the value, and turning it anticlockwise will always decrease the value, wherever the pot happens to be.

Pressing the right pot causes the parameter value to jump directly to the position of the knob, bypassing the soft takeover. This can be useful, say, if you want to set a number of parameters to a similar value. For example, if you wanted to set a number of mixer channel gains to the same value, you could turn the right pot to set the first value, then step through the other channels with the centre pot and press the right pot to jump the values to the right area, without having to turn the pot a considerable distance for each parameter.

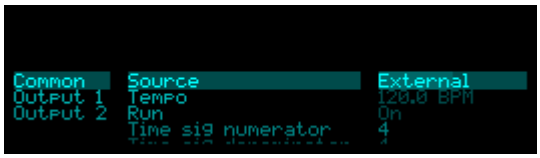
Pressing the right encoder sets the parameter to its default value, with some exceptions:

- For parameters with a ‘Confirm’ function, for example the wavetable selection of the Poly Wavetable algorithm, pressing the right encoder confirms the new value.
- For string parameters, pressing the right encoder switches to a view where you can edit the string. See the notes on editing text, above.
- For parameters specified in dB, pressing the right encoder sets the parameter to its default, or if the parameter is already at minimum (usually -∞dB), to 0dB.
- For parameters specified in %, if the parameter is at 0%, pressing the right encoder sets it to 100%, and if it’s at 100%, to 0%.

Pressing button 4 is a shortcut to the Mappings menu. If you prefer, you can change this in the Settings to go directly to one of the CV, MIDI, or I2C Mappings submenus instead.

Press the left encoder or the left pot to switch to the algorithm overview. You can also hold down the left encoder or left pot and turn it while held to select a different algorithm, in which case releasing the encoder or pot returns you to the parameters view. This is a shortcut to what would otherwise be a press to switch view, a turn, and another press to switch the view back.

Sometimes you may see parameters “greyed out”. This indicates that the parameter is currently unused, most likely because of some other parameter selection. For example, the tempo parameter of the Clock algorithm is greyed out if the algorithm’s internal clock is not being used.



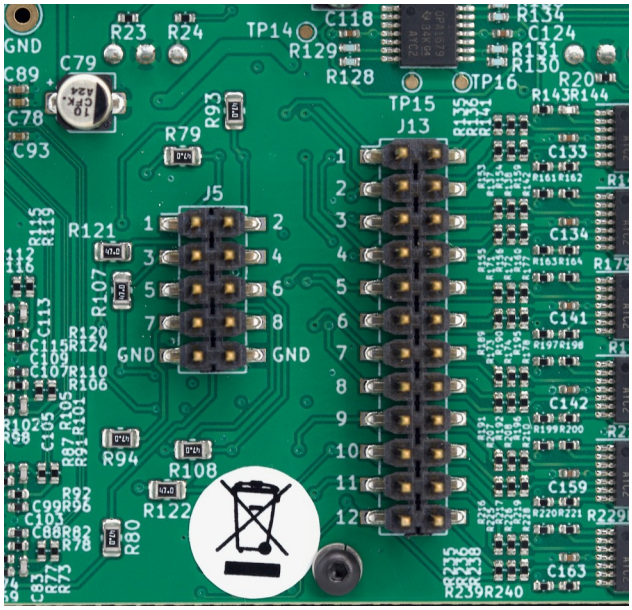
Inputs and outputs

The disting NT's input and output jack sockets are illuminated, lighting red for positive voltage and blue for negative voltage. (Audio-rate signals appear purple, since you see a rapid alternation of positive and negative.)

The sockets are 3.5mm TS jacks. Do not use TRS cables.

The inputs and outputs are all DC coupled.

All of the inputs and outputs are also available on headers on the rear of the module. The outputs are on J5 and the inputs on J13:



Inputs

The twelve sockets on the left of the module (numbered in black on white) are the inputs.

The inputs can operate at either modular or line level, selected by jumpers. See the image above. The left pins on the J13 header correspond to the front panel sockets, and operate at modular level. The right pins are line level. If you fit a jumper across a pair of pins, this shorts the input socket to the line level input i.e. it makes the front panel socket a line level input.

Mode	**Maximum voltage before clipping**	**Input impedance**
Modular	±12V	101kΩ
Line	±3.5V (+10dBu)	29kΩ

Outputs

The six sockets on the right of the module (numbered in white on black) are the outputs.

The outputs have a maximum range of ±11V.

There are an additional two outputs available via the J5 header on the rear of the module. See the image above. The pins of the header are numbered according to the output numbers; pins 1-6 correspond directly to the front panel output sockets, while pins 7-8 are the two additional outputs. Ground ('GND') connections are also exposed on this header.

One possibility is to connect the additional two outputs to two sockets on the included breakout module - or even to connect them to a single socket, as a stereo TRS output.

USB

The module has a type C USB socket to the left of the display. The disting NT is a High Speed USB 2.0 device. It is not a USB host.

The module does not draw power from the USB connection.

If you are planning on using the USB socket more than occasionally, we recommend buying a cable with a right-angled plug, to keep the cable out of the way of the display.

The disting NT is a MIDI Class, Audio 2.0 Class, and Video Class compliant device. As such it needs no drivers when used with macOS, iOS, and Linux. An audio driver for Windows is available via our website[17].

The USB socket can also be used to mount the module's MicroSD card as a removable drive on your computer – see below.

MIDI connections

The disting NT can send and receive MIDI via a breakout header, via the Select Bus, and via USB.

MIDI breakout

The disting NT's MIDI breakout header is a four pin connection, exactly the same as on all of our other modules with MIDI – at the time of writing, these are the disting mk4 and EX, the FH-2, the ES-9, and the CVM-8. The header can be directly connected to DIN or TRS sockets – for example, our own MIDI breakout modules, or the sockets built into certain Eurorack cases.

17 https://expert-sleepers.co.uk/downloads.html

The wiring is as follows:

Breakout pin	MIDI DIN pin
1	OUT pin 4
2	OUT pin 5
3	IN pin 5
4	IN pin 4

Pin numbers refer to the 5-pin DIN socket as in the MIDI specification[18].

As well as connecting the module to a breakout, you can also use the header to connect directly to another of our modules. In this case, connect pin 1 on one module to pin 4 on the other, pin 2 to pin 3, and so on, so that the outputs connect to inputs and inputs connect to outputs.

Select Bus

The Select Bus is a means of inter-module communication currently supported by a handful of modules from various manufacturers, including the Malekko Varigate 8+, Macro Machines Storage Strip and the Make Noise Tempi & René. It is essentially a MIDI connection piggybacked on what was originally conceived as the CV bus, which is one of the lines on the power bus that connects all modules in the system.

The ‘Select Bus protocol’ specifies certain messages to be used for preset saving and recall, which are combinations of MIDI program change and CC messages. The disting NT currently does not implement this protocol at all – it simply uses the Select Bus as a MIDI connection. It is particularly convenient since no additional cables are required – the modules are connected simply by virtue of being on the same power bus.

USB MIDI

The disting NT is a MIDI Class compliant device, as noted above.

MicroSD card

The card slot is just to the left of the left pot. Insert cards with the exposed contacts facing right. It is safe to insert a card after the module is powered up.

Do not remove the card while the module is writing to it, or data corruption could result.

If you do remove the card and re-insert it while the module is powered on, you will need to manually re-mount it via the menu. See the Misc menu, below.

18 https://midi.org/5-pin-din-electrical-specs

Card format

The disting NT requires the card to be formatted as FAT32 or exFAT. If you need to format a card, we recommend that you use the formatting tool provided by the SD Association, which you can download here[19].

Card content

Nothing on the card is actually required for the module to operate; you can insert a completely blank card (or not insert a card at all) and the module will function.

Note though that, in the current firmware, a card is required for storing presets, so you will probably want to keep a card installed at all times.

The card provided with the module is preloaded with some content created by ourselves and our partners, including

- Spitfire Audio[20] - multisamples, including the iconic Soft Piano.
- Goldbaby[21] - samples, mostly drums.
- Adventure Kid[22] - wavetables.

Card layout

Data on the card is organised into a number of top level folders to keep things tidy. While some functions (e.g. preset loading) allow you to select files from anywhere on the card, many of the core algorithm functions require data to be in a specific place – e.g. the algorithms that play samples require all the samples to be in the ‘samples’ folder.

The standard folders, as found on the card supplied with the module, are as follows:

FMSYX	FM banks (SysEx format, usually .syx)
kbm	Scala keyboard map files (.kbm)
MIDI	Subfolders, each containing MIDI files (.mid)
MTS	MIDI Tuning Standard SysEx dumps (.syx)
presets	The default folder into which presets are saved. Can be changed in the Settings menu.
programs	Subfolders contain programs for specific algorithms.
programs/lua	Lua scripts for the ‘Lua Script’ algorithm (.lua)

19 https://www.sdcard.org/downloads/formatter/

20 https://www.spitfireaudio.com

21 https://www.goldbaby.co.nz

22 https://www.adventurekid.se

programs/plug-ins	C++ plug-ins (.o)
programs/three_pot	Programs for the ‘Three Pot’ algorithm (.3pot)
recordings	The default folder for audio recordings made by the Audio Recorder algorithm. Can be changed in the Settings menu.
samples	Subfolders, each containing audio files (.wav)
scl	Scala scale files (.scl)
ui_scripts	Scripts for custom UIs (.lua)
wavetables	Wavetables, either as single file wavetables, or subfolders of single cycle waves (.wav, either way)

FM banks

The ‘FMSYX’ folder on the card contains Yamaha DX7 format voice banks in SysEx format. Such files usually have the extension ‘.syx’ and will be exactly 4104 bytes in size. Literally hundreds, if not thousands, of voice banks can readily be found online, or you can make your own with various software tools designed as DX7 patch editors.

In the current firmware, the FM banks are primarily used by the Poly FM algorithm, and up to 100 banks can be used at once.

Scala files

Many algorithms on the disting NT support microtuning via the common Scala[23] file formats. Scale files (.scl format) go in the ‘scl’ folder and keyboard map files (.kbm) format go in the ‘kbm’ folder.

Keyboard map files are not required – the algorithms can apply microtuning from a .scl file only – but are supported for more detailed control over tone mapping if desired.

Scala files can be generated by the Scala application, by another tool that writes Scala-format files (for example, Scale Workshop[24]), or even written by hand in a text editor.

In the current firmware, you can use up to 1000 .scl files and 1000 .kbm files.

MTS (MIDI Tuning Standard)

MTS can be used live, over a MIDI connection, or via SysEx dump files. The disting NT supports reading MTS bulk SysEx dumps from the card. Such files will be exactly 408 bytes in size, and should be put in the ‘MTS’ folder.

In the current firmware, you can use up to 1000 MTS SysEx dumps at once.

23 https://huygens-fokker.org/scala/

24 https://sevish.com/scaleworkshop

MIDI song files

Some algorithms, notably the MIDI Player, support MIDI song files. Such files typically have the extension '.mid'.

Song files should be put in subfolders of the top level 'MIDI' folder (i.e. not directly within the 'MIDI' folder itself).

A good technical description of the MIDI file format is here[25]. Most DAWs export MIDI files in some form – for example, Ableton Live exports MIDI clips as single track (format 0) MIDI files, while Logic Pro will export multi-track (format 1) files. And of course, the internet is full of sites where you can download complete songs or looping patterns in MIDI format.

Those so inclined might find it useful to create MIDI files programmatically, which can be easily done, for example, in Python using libraries such as mido[26].

The current firmware supports up to 1000 folders of 1000 files each.

Presets

The disting NT uses JSON[27] format for presets. As such they are human readable, and if necessary can be manipulated using a simple text editor, though this is not something we expect most users to do.

Presets can be loaded from anywhere on the card, so you can if you wish organise them into folders. However they are always saved to a location that is chosen in the Settings, which defaults to a top level folder named 'presets'.

Samples

Algorithms that play samples look for them in the 'samples' folder, which should contain subfolders, each of which contains audio files in standard WAV format. Subfolders may be nested.

Mono or stereo files are supported, at any sample rate. Supported bit depths are 8 bit, 16 bit, 24 bit, and 32 bit float.

The current firmware supports up to 1000 sample folders and a total of 10,000 samples, with a maximum of 1000 files per folder.

Q: Why is there a maximum at all? Can't I just load a file from the card and play it?

A: Scanning the card and pre-preparing lists of samples means that sample selection can be mapped to a CV (or MIDI CC etc.). One of the design goals of the disting series has always been to integrate well into a modular synth, not simply to be physically a Eurorack module.

The card is scanned for samples at power on, and most of the salient data cached back onto the card, so a slow scan of each folder is only done when it's added or changed.

25 http://www.music.mcgill.ca/~ich/classes/mumt306/StandardMIDIfileformat.html

26 https://mido.readthedocs.io/en/stable/

27 https://en.wikipedia.org/wiki/JSON

Avoid turning off the module while the scan is in progress, since it's potentially writing to the MicroSD card.

Sample naming

The sample naming convention is the same as for our disting EX[28] module, so if you have libraries prepared for that module they can be easily shared by the disting NT. Look at the samples on the card provided with the module for some examples. The filenames are examined for various patterns, all starting with an underscore ('_'), to set various properties of the files. These are as follows:

Key	Example	Description
_<note name>	piano_A2.wav piano_C#5.wav	Sets the natural note pitch, that is, the pitch of the audio in the sample as recorded. Use naturals and sharps ('#') only, not flats.
_SW<note number>	piano_A2_SW40.wav	Sets the sample's switch point when used in multisample algorithms. The 'note number' is the MIDI note number, in decimal. The switch point specifies the lowest pitch that will use the file.
_RR<number>	snare_RR1.wav snare_RR2.wav	Links files as round-robin variants of the same sample.
_V<number>	snare_V1.wav snare_V2.wav	Sets up a velocity switched sample. The lowest-numbered _V is used for the lowest velocity.

If round-robins and velocity switches are combined, the velocity switch comes first e.g.

snare_V1_RR1.wav, snare_V1_RR2.wav
snare_V2_RR1.wav, snare_V2_RR2.wav

In other words, each velocity layer can have round-robins.

Automatic 'natural' values

If sample files are not assigned a natural pitch (for example, a folder of drum samples), they are assigned values from 48 upwards, so playing a keyboard chromatically from C3 will play a new sample on each semitone.

Automatic 'switch' calculation

If the switch point for multisample files is not explicitly specified, the disting NT calculates sensible defaults, as follows.

If the gap between neighbouring samples is at most 3 semitones, the switch is set so that the higher sample is pitched down within the gap.

For larger gaps, the lower sample is pitched up over half the remaining range.

28 https://expert-sleepers.co.uk/distingEX.html

For example:

Gap between samples (semitones)	Behaviour
1	Higher sample stretched down over 1
2	Higher sample stretched down over 2
3	Higher sample stretched down over 3
4	Higher sample stretched down over 3; lower sample stretched up over 1
5	Higher sample stretched down over 4; lower sample stretched up over 1
6	Higher sample stretched down over 4; lower sample stretched up over 2
	Etc.

Converting from other sample library formats

There are tools in our Github repository[29] for converting the output of Logic Pro's autosampler and of DiscoDSP's Bliss to a format readable by the disting EX (and by extension the disting NT).

We also have a tool for converting SoundFont®s which you can find here[30].

Our format is supported by the very capable ConvertWithMoss[31], which supports many other formats.

Creating your own samples

Creating new sample sets is simply a matter of collecting WAV files in a folder on the MicroSD card and naming them appropriately, if required (see above). Non-pitched samples e.g. drums need no special naming.

There are tools available to automate the sampling of existing instruments, for example DiscoDSP's Bliss as already mentioned.

The disting NT's own Auto-sampler algorithm will automatically sample any instrument it can trigger via CV/gate or MIDI. The files it records to the MicroSD card are named correctly for use by the multisample playback algorithms.

Loop markers in WAV files

The disting NT supports reading loop information embedded in the WAV file. If this information is not present, any playback that loops the sample simply loops the whole file.

The disting NT looks for 'cue ' chunks and 'smpl' chunks. Which of these your files contain will

29 https://github.com/expertsleepersltd/distingEX_tools

30 https://github.com/expertsleepersltd/sf2_to_dex

31 https://mossgrabers.de/Software/ConvertWithMoss/ConvertWithMoss.html

depend on the authoring software.

When interpreting a 'cue ' chunk, loops are inferred either from markers or regions, as follows:

1 marker point in file	Marker is assumed to be loop start; loop is from the marker to the end of the sample.
2 marker points in file	Markers are used as loop start and end.
3 or more marker points in file	First marker is ignored (assumed to be playback start point); second and third markers used as loop points. Remaining markers ignored.
1 or more regions in file	First region is used as the loop; other regions and markers ignored.

When interpreting a 'smpl' chunk, the first loop in the chunk is used, and any others are ignored.

There are some example multisamples with loop points in our GitHub here[32].

Round robin mode

Algorithms that have a 'Round robin mode' parameter allow you to choose how round-robin variants of the same sample are chosen. The options are as follows:

Sequential	The round-robins are played in numerical order.
Random	The round-robins are played in a random order.
Random2	The round-robins are played in a random order, except that the same round-robin cannot be played twice in a row.
Random3	The round-robins are played in a random permutation, then another random permutation, and so on.

Wavetables

A number of algorithms (e.g. Dream Machine and Poly Wavetable) include wavetable-based oscillators. These look for wavetables inside the ‘wavetables’ folder.

Within the 'wavetables' folder, wavetables can take one of two forms: a single WAV file containing all the waveforms concatenated, or a folder of WAV files, one per waveform. All of the examples included on the card use the latter format.

Waveform WAVs can use any supported bit depth. The sample rate is unimportant, since the file is assumed to contain exactly one cycle and so can be pitched arbitrarily.

When using a folder of single waveform WAV files, each file should be the same length. For example, in the ‘AKWF_0001’ wavetable on the MicroSD card each WAV file is 600 samples long.

32 https://github.com/expertsleepersltd/distingNT/tree/main/samples

When using a single concatenated WAV file, the disting needs to know how many frames in the file make up one waveform. It does this as follows:

- If the filename ends in a dash followed by a number, that number is taken to be the number of frames in one waveform. For example, if the filename is “awesome-256.wav”, then the disting will interpret the file as having 256 frames per waveform.
- If the number of frames in the file is a multiple of 2048, then the disting assumes 2048 frames per waveform. This is a convenience to support the popular ‘Serum’ format of wavetable, since those always have 2048 frames per waveform. This means you can simply download one of thousands of Serum wavetables from the internet and drop them straight onto the disting NT’s MicroSD card.
- If neither of the above apply, the disting NT assumes 600 frames per waveform.

If you have a wavetable that you’d like to use but you’re not sure of the correct number of frames per waveform, open the file in a sample editor, or use the waveform view in your DAW. It should be fairly clear where the waveforms lie just from looking at it. Measure the number of frames in one waveform, or count the number of waves in the file and divide that into the total length of the file.

The current firmware supports up to 1000 wavetables on the MicroSD card. The maximum size of each individual wavetable depends on the specific algorithm using it.

Signal flow

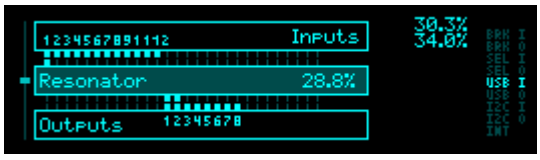
The disting NT implements a very straightforward scheme for passing signals around the system.

All algorithms process data from, and to, a selection of ‘busses’[33], of which there are 64. The algorithms get to work on this data one at a time, in sequence, from top to bottom as seen in the algorithm overview.

At the top of the chain, the first 12 busses are driven from the module’s 12 inputs. The remaining 52 busses are empty/silent.

The next 8 busses feed the module’s 8 outputs, at the end of the chain. The remaining 44 busses are ‘aux’ busses, provided for more routing flexibility between algorithms. Note that only the first 8 aux busses are shown in the display; the full 44 can be visualised in the preset editor tool (see below).

In the algorithm overview, each algorithm’s block has a graphical representation of which busses it is using for input and output:



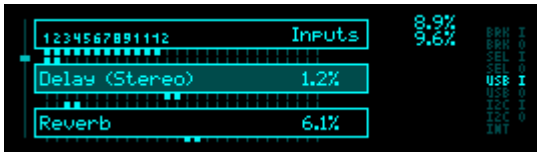
For example, in the image above, the Resonator is taking input from the “Input 1” bus and producing output on the “Output 1” & “Output 2” busses.

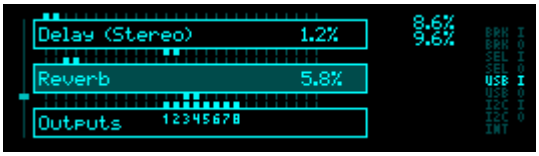
33 We’re using the term ‘bus’ as commonly used when describing audio mixers: see https://en.wikipedia.org/wiki/Audio_bus

Algorithms can be independent

The module is not restricted to running a single signal chain, from inputs to outputs. Implicit in the above bus scheme is the fact that algorithms can work on any bus, no matter where they are in the stack.

To take a simple example, the first algorithm could take inputs from physical inputs 1-2 and output to physical outputs 1-2, while the second algorithm could take inputs from physical inputs 3-4 and output to physical outputs 3-4 – thus operating as two completely independent processes; two separate virtual modules, if you like.





Output mode

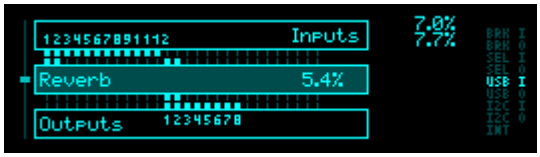
Many algorithms have a parameter (or parameters) named ‘Output mode’, which can either be ‘Add’ or ‘Replace’. This refers to whether the algorithm output in question will be summed onto the output bus, or will replace the bus contents entirely. Both are useful in different situations.

For example, algorithms which are instruments will typically add their outputs to the bus, so if you have multiple instruments feeding the same bus, their outputs will all be audible together.

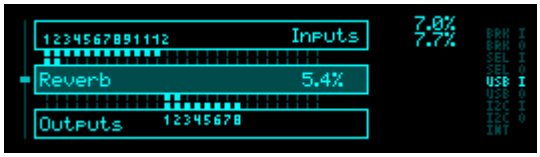
On the other hand, an EQ will typically want to replace the signal on the bus with the EQ’d version.

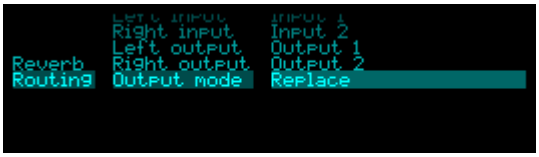
For other algorithms, the two modes are both useful in different usages. For example, a delay might want to add its output to a master mix bus when used as a send effect, but might want to replace the bus signal when used as an insert (then perhaps using its own wet/dry mix to control the signal balance).

In the algorithm overview, outputs which are using ‘Add’ mode will also show as active inputs to the block:









Algorithm specifications

Algorithms have the concept of ‘specifications’. These are in general either

- Things that affect the resource (chiefly memory) usage of the algorithm, for example, the maximum delay time of an echo effect.
- Things that affect the number of parameters an algorithm has, for example, the number of

audio channels of a mixer.

Some algorithms have no specifications; currently, the highest number of specifications that any algorithm has is three.

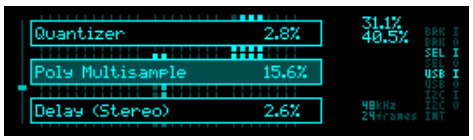
Specifications are set when initially adding an algorithm. If necessary, they can be changed later via the ‘respecify’ menu.

Changing specifications interrupts audio, as it’s essentially causing the entire preset to be rebuilt.

Sample rate and buffer size

By default, the module runs at a sample rate of 48kHz, with an internal buffer size of 24 frames. This means that all signal processing into, within, and out of the module happens at 48kHz, and that “control rate” processing (which includes parameter mapping, and MIDI handling) happens every 24 frames i.e. at 2kHz.

The current sample rate and buffer size are shown in the bottom right of the algorithm overview:



You can change the sample rate and buffer size (from a fixed selection) via the Settings menu (below). In general a higher sample rate or smaller buffer size increases CPU load.

When using USB audio, you can choose (via the Settings) to allow the USB host to control the sample rate (for example, via your DAW’s audio preferences).

CPU usage/overload

The CPU usage of each algorithm is shown in its box in the algorithm overview. The total CPU usage for all algorithms is shown top right. You should aim to keep this below about 90% for reliable operation.

Below this figure is another percentage, which is an estimate of the CPU usage of the entire module, which takes into account MicroSD card access and other background processes. In some cases, you may find this goes significantly higher than the algorithm CPU usage.

If you add too many complex algorithms, or if you adjust algorithm parameters in certain ways, you may end up in a situation where the module’s CPU can no longer keep up. This will likely result in crackly audio and the UI may run slowly or even freeze.

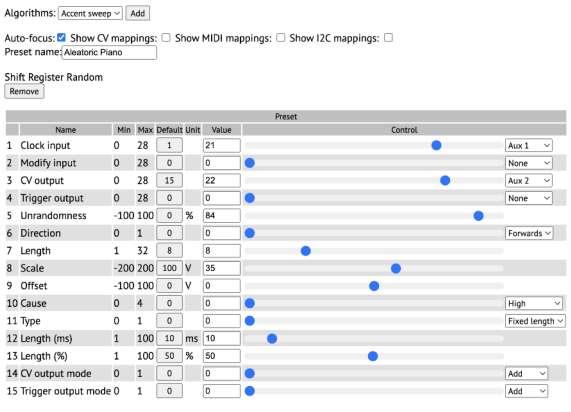
If it detects this situation, the module will suspend all algorithm processing and mute the outputs. The algorithm overview will show the message “CPU OVERLOAD !MUTED!”.

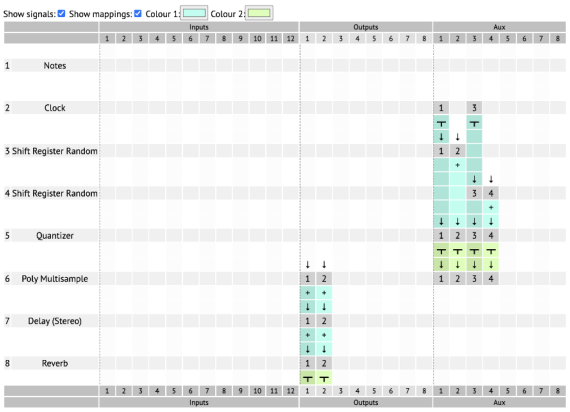
The module will stay muted until you do something which is likely to modify the CPU usage, which includes:

- Removing an algorithm.
- Respecifying an algorithm.
- Bypassing an algorithm.
- Loading a preset or simply doing ‘New preset’.

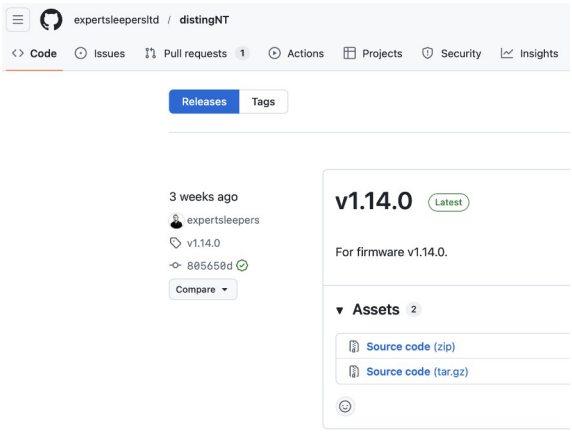
Preset editor/routing analyser

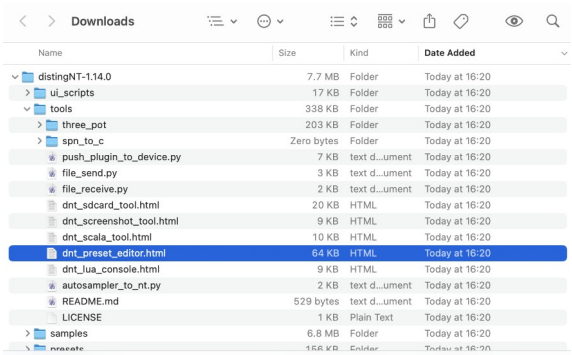
There is a web-based tool in our GitHub repository (here[34]) for editing presets. This is very much a work-in-progress, but useful nonetheless. As well as allowing you to view/edit algorithm parameters, it will show you an analysis of the routing within the preset.





You can install the tool by visiting the releases page of the GitHub repository here[35]. Find the latest release (or the one that corresponds to the firmware version you're using) and click on the zip file to download it. In the zip you'll find a 'tools' folder, and in there 'dnt_preset_editor.html'. Drag this into Google Chrome (it has to be Chrome as most browsers don't support the required Web MIDI API).





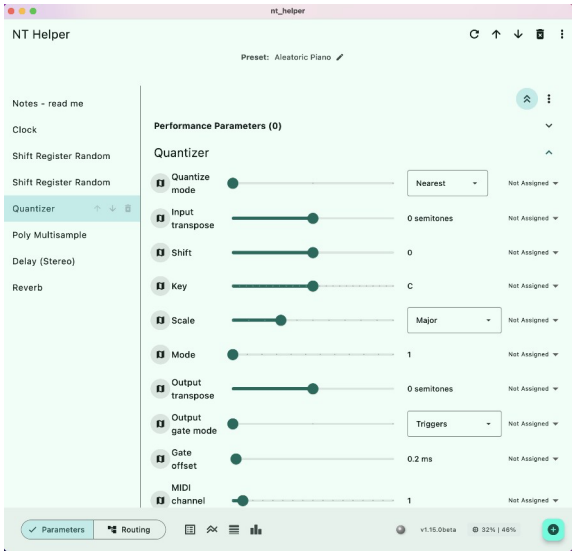
34 https://github.com/expertsleepersltd/distingNT

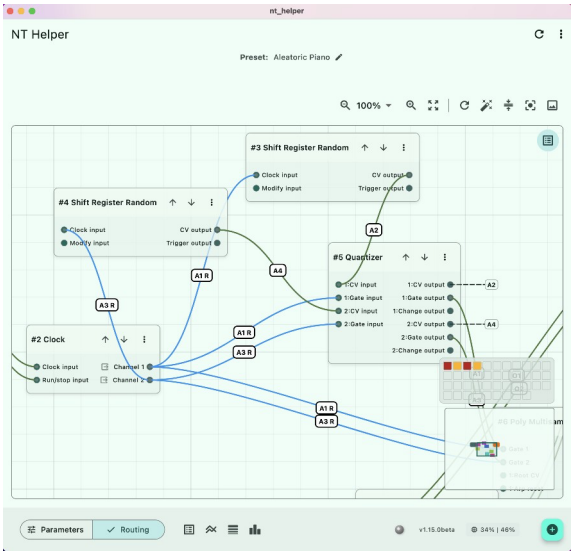
35 https://github.com/expertsleepersltd/distingNT/releases

NT Helper

In addition to the tool above, there is also a third party tool called “NT Helper”, which was written by a user of the module. It runs on Android, Windows, Linux, macOS, and iOS, and can be found here[36].

Please note that this tool is supported by its author, not by Expert Sleepers. You can get support from the author via the Expert Sleepers Discord server[37], in the #nt-helper-app channel.





36 https://nosuch.dev/nt-helper/

37 https://modwiggler.com/forum/viewtopic.php?t=287914

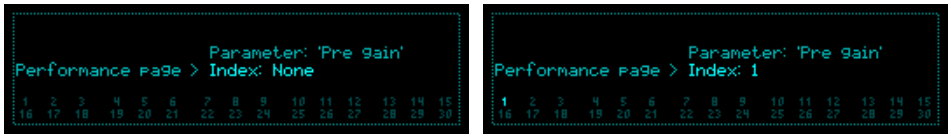
Performance page

The ‘performance page’ is a somewhat experimental feature, designed to allow quick access to key parameters within a preset from a single location – allowing you to tweak and perform with your preset without navigating around between algorithms. User feedback on this feature is particularly welcome, to help us flesh it out more for future firmware versions.

The performance page also includes knob recorders for instant gestural looping.

Adding parameters to the performance page

To make a parameter appear in the performance page, assign it an index via the Mappings menu (below).



The numbers across the bottom of the screen show the possible indices, with the ones in use highlighted.

As soon as any parameter in the preset has a performance page index, extra lines will appear in the overview screen at the very top and bottom:



These two are the same, and are duplicated simply to increase the ease of getting to one or the other quickly.

The interface for navigating into and within a performance page is very close to that of an algorithm – e.g. press the left encoder to enter the page.



The left column shows the algorithm name; the middle column shows the parameter name; and the right column shows the parameter value. Scrolling between parameters and editing the parameter value all use the same controls as the usual algorithm parameter interface.

Because there is limited space, the left column showing the algorithm name may be truncated. Remember that you can rename algorithms via the Algorithms menu (below).





The parameters are sorted in the list according to their performance page index, as assigned via the Mappings menu. If two (or more) parameters have the same index, they are further sorted by the index of their algorithms within the preset.

Custom performance page UI

Pressing button 2 (the same button that switches an algorithm's view to its custom UI) switches to a view where the performance page parameters are arranged in pages of 3, with each assigned to its own knob:



In this view, the three pots control the three parameters on the page, and the left encoder switches between pages.

There are two options for how the pages are laid out according to the performance page indices, which is set via the Settings menu (below). The default, 'Condensed', uses the indices only to order the parameters – they are grouped into threes to make the smallest number of pages. The other option, 'As indexed', attempts to use the indices as absolute locations, allowing for gaps – for example if the three parameters shown above had the indices 1, 4, and 5, the layout would be as below:





Knob recorders

In the custom view just described, with the parameters in groups of three, each pot also has a knob recorder function. This is very much inspired by the simple yet effective knob recorder on the disting mk4.

Press a pot and hold it in while turning to activate the knob recorder function. The display will show 'Record' and an incrementing timer below the value:



When you release the pot, it will immediately start playing back the knob movement you recorded, in a continuous loop (the display shows ‘Play’):



To stop playback and regain manual control, simply turn the pot.

Note that you can switch to another page of the performance page, or even leave the performance page altogether, and any playing knob recorders will continue to play.

Menu reference

The pages that follow describe the disting NT’s menus.

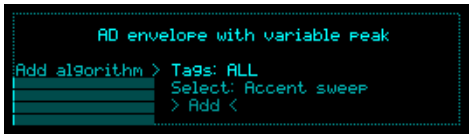
Algorithms menu

The Algorithms menu is where you interact with the list of algorithms that makes up a preset – adding/removing algorithms, reordering them etc.

In general the operations here apply to the ‘current’ algorithm, that is to say, the one highlighted in the algorithm overview.

Add algorithm

Allows you to select an algorithm, which will be added to the preset.



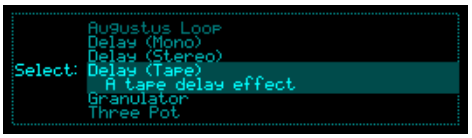
The “Tags” submenu allows you to filter the available algorithms, to show only certain categories e.g. instruments, effects, or utilities.

You can turn the right encoder to select the algorithm directly, and press the right encoder to add it.

Adding an algorithm with the right encoder, or with the left encoder on the ‘> Add <’ menu, adds the algorithm to the end of the preset. Pressing the centre pot instead inserts the algorithm in front of the current algorithm.

You can also press the left encoder to go down to a more detailed view, which shows a description of each algorithm as well as its name.





If the algorithm has specifications, you can adjust these here before adding it.

To the left of the display you’ll see four memory usage gauges, showing the impact that the new algorithm will have on the four memory domains. The memory usage of the algorithm may, or may not, be affected by its specifications. If the algorithm exceeds the available memory, the message ‘INSUFFICIENT MEMORY’ will be shown, and you won’t be able to add it.



If you hold button 4 (‘Shift’), you’ll see that ‘Add’ changes to ‘Add bypassed’. This adds the new algorithm with its Bypass parameter already enabled.

You can add a total of 32 algorithms to a preset. This limit is completely arbitrary. If you feel it should be higher, please let us know – and we want to see that massive preset you want to make!

Respecify algorithm

Allows you to change an algorithm's specifications, while maintaining its parameters and position in the list.

As in the 'Add algorithm' menu, a memory gauge is shown. You cannot respecify an algorithm if doing so will cause its memory usage to exceed that available.

Respecifying an algorithm may add or remove parameters (for example, respecifying the mixer will add or remove parameters to reflect the change in the number of mixer channels). Any added parameters will assume their default values.

Remove algorithm

Removes the current algorithm from the list.

Move algorithm up

Moves the current algorithm up one position in the list.

Move algorithm down

Moves the current algorithm down one position in the list.

Customise name

Allows you to change the algorithm's name. By default, each algorithm takes the name of the algorithm of which it is an instance e.g. if you add a 'Reverb' the algorithm in the list will be called 'Reverb', but you can change this to anything you like. This can be especially useful if you have more than one instance of the same algorithm, so you can quickly tell them apart.

Duplicate algorithm

Duplicates the current algorithm, if available memory permits. The duplicate algorithm is positioned immediately after the original in the list.

Presets menu

The Presets menu is where you load and save presets, and perform related functions.

A preset contains the full state of the module – the algorithms, their parameters, and any mappings. Presets are stored on the MicroSD card (in this firmware version, at least – future versions may offer the possibility of storing presets in flash memory).

Load preset

Lets you choose a preset file from the MicroSD card to load.

Name preset

Lets you set the preset name. This will be used as the filename when the preset is saved. See the notes on editing text, above.

Generate random preset name

Sets the preset name to a randomly generated phrase.

Save preset

Saves the current preset in memory to the MicroSD card, using the name set in ‘Name preset’ and to the folder chosen in ‘Set save location’.

New preset

Removes all algorithms and sets the preset name back to the default ‘Init’.

Split/combine

This submenu contains a number of functions to allow you to combine presets, or to split parts of presets out for reuse elsewhere.

Append preset

Like “Load preset”, but the chosen preset file is added to the current state, instead of replacing it – that is, the algorithms in the preset file are appended to the list of running algorithms (if possible, that is, if memory permits).

Insert preset

Like “Append preset”, but the file is inserted in the existing preset before the current algorithm.

Prepend preset

Like “Append preset”, but the file is inserted at the top of the existing preset instead of at the bottom.

Save single algorithm preset

Saves the current algorithm only to a preset file. Useful in combination with ‘Append preset’ to copy a single algorithm from one preset to another, or simply to save a library of presets for a particular algorithm, independently of whatever else is going on in the preset.

The preset file is named using the algorithm’s default name and its custom name as set via the ‘Customise name’ menu. For example, if you have an instance of the ‘Reverb’ algorithm and you’ve

customised its name to ‘Hall’, the preset will be saved as ‘Reverb - Hall.json’.

Save range

Similar to ‘Save single algorithm preset’, but saves a range of algorithms from the current preset to a new file.

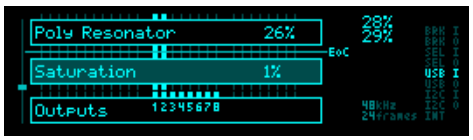
The saved file is named using the current preset name, plus numeric suffices to indicate the range of saved algorithms. For example, if your preset is named ‘elegiac’ and you save algorithms 3 thru 5, the filename will be ‘elegiac-3-5.json’.

End of chain

The disting NT supports the concept of an “end of chain” (hereafter EoC) preset. This is a notional split in the algorithm list, where everything after a certain point stays the same when presets are loaded. This is designed to be particularly useful for setting up final bus compression, EQ etc. so you could for example arrive at your gig, tweak the EQ for the venue, and then have that stay the same as you load the various presets used during your set.

Note that there’s nothing special or different about an EoC preset file – it’s just a regular preset file containing a list of algorithms and their parameters.

The EoC marker is shown in the overview page as a line with “EoC” at the right side:



The various EoC submenu functions are as follows.

Load EoC

Lets you choose a preset file to load as an EoC preset. If there is no existing EoC preset, the chosen file is appended to the preset, and the EoC marker is set. If there is an existing EoC preset, everything below the EoC marker will be removed and replaced by the loaded file.

Name EoC

As ‘Name preset’, but used when saving the EoC preset.

Generate random name

Sets the EoC preset name to a randomly generated phrase.

Save EoC

Saves a preset file using the name set under ‘Name EoC’, containing everything below the EoC marker.

Remove EoC

Removes all algorithms below the EoC marker, and resets the marker so there is no EoC section.

Adjust

Lets you move the EoC marker up and down in the algorithm list. The bottom of the screen shows the name of the algorithm that the marker will be above.



Set save location

Lets you choose the folder on the MicroSD card where presets will be saved. Note that presets can be loaded from anywhere – this only affects the save operation. You're free to take the MicroSD card over to a computer to rearrange, rename, or archive the saved preset files.

Load disting EX preset

Lets you choose a disting EX preset file and attempt to load it as a disting NT preset. At the time of writing, this will only succeed for disting EX 'Poly Wavetable' presets. If you have disting EX presets for other algorithms that you particularly would like to load into the disting NT, please get in touch!

Author name

Lets you set a name that will be saved in presets as the "author".

Mappings menu

Video[38]

The Mappings menu is where you set up remote control of algorithm parameters by CVs, MIDI, and I2C. Any parameter can be controlled by any combination of these. Also, any control source can control as many algorithm parameters as you like.

The mappings shown refer to the current algorithm, highlighted in the algorithm overview.

When you enter the menu, the various mapping submenus will reflect the current parameter that was being edited in the single algorithm view. Likewise, if you select a different parameter while setting up the mappings, that parameter will be the current one when you leave the menu.

Mappings are part of the preset, and are saved and loaded with it.

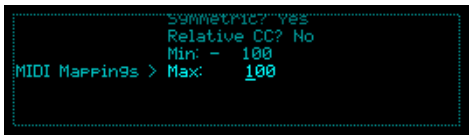
38 https://www.youtube.com/watch?v=utpF6fIP_zM

Editing mappings values

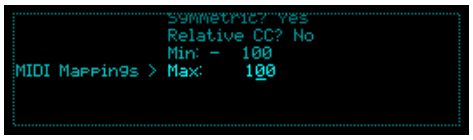
The mappings UI includes a unique method of adjusting values which would otherwise be very fiddly to adjust using the standard knob/encoder UI used elsewhere in the module.

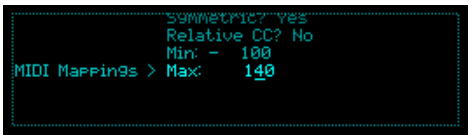
The values in question are the Delta for CV mappings, and the Min & Max for MIDI and I2C mappings. (See below for details of what these values actually mean.)

Taking the MIDI mappings Max as an example:



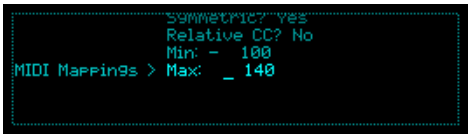
You will see that there is a cursor under the value, which is used to select a digit to adjust:

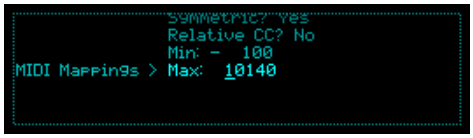




Move the cursor with the middle pot; adjust the digit with the right pot.

You can position the cursor on an empty digit and turn the right pot, to jump directly to a large value:





You can also use the right encoder as normal to adjust the value.

CV Mappings

This is where you set up a mapping from a CV source. This will commonly be an external CV applied to one of the module's inputs, but can also be any signal on any bus within the system – so you can for example run an LFO algorithm and map the CV that it generates to control a parameter on another algorithm.

Unless the mapping is defined as a 'Gate', the mapping *adds* to the base parameter value set in the preset – it does not *replace* it. To put it another way, a CV of 0V, or a disconnected cable, will always give you the parameter value as if it wasn't mapped.

There are three choices for where the CVs to be used for mapping come from.

The first, and default, is 'Own inputs'. This means that the algorithm will look at state of the busses at its own inputs for the mapping CVs.

The second is 'Module inputs' – the mapping CVs are taken from the physical inputs on the module.

The third option is that the mapping CVs are taken directly from the outputs of another algorithm in

the preset.

There is some redundancy here, inasmuch as if you choose the source as the algorithm immediately before the mapped algorithm in the preset, the effect is the same as if the source was ‘Own inputs’. However, these will differ in terms of what happens if the preset is reordered or algorithms are deleted.

The items that can be set up per mapping are as follows:

Parameter	Chooses the parameter to map.
Source	Chooses the CV source – ‘Own inputs’, ‘Module inputs’, or the outputs of one of the algorithms in the preset.
Bus	Chooses the CV source bus.
Unipolar?	If set, the CV is clamped to 0V (i.e. negative voltages are treated as 0V).
Gate?	If set, the CV can only toggle the parameter between its minimum and maximum values.
Volts, Delta	These two together determine the relationship between CV voltage and parameter value. The value added to the parameter is equal to (CV voltage) x (Delta) ÷ (Volts) To put it another way, a CV of (Volts) is required to change the parameter value by (Delta).

MIDI Mappings

This is where you set up mappings from MIDI CCs (Continuous Controllers), channel pressure (aka aftertouch), and notes.

Unlike CV mappings, MIDI mappings directly set the algorithm parameters – they do not add to them.

While in the MIDI Mappings menu, pressing the right encoder enters “Learn” mode. While this is active, the module listens for MIDI CCs and notes, and the first one received will be set as the source for the current mapping. Press the right encoder again to leave Learn mode without altering the mapping. Note that there is a setting for whether setting up a mapping via Learn also disables any other mappings that were already using the CC.

14 bit CC pairs are supported. This is a convention used by some controllers where a high resolution value is split across two CCs: a ‘high’ or ‘most significant’ byte using a CC in the range 0-31, and a ‘low’ or ‘least significant’ byte using the CC 32 higher (i.e. in the range 32-63). Unfortunately nobody can agree which should be transmitted first, so we support both. When using 14 bit CCs, specify the CC in the range 0-31 for the CC number for the mapping.

The items that can be set up per mapping are as follows:

Parameter	Chooses the parameter to map.
Channel	Sets the MIDI channel for the mapping controller.
Type	One of • ‘CC’ • ‘Note – momentary’ (note on sets the Max value; note off sets the Min value) • ‘Note – toggle’ (note on toggles between Min and Max; note off is ignored) • ‘14 bit CC - low first’ (a 14 bit CC where the low byte is sent first) • ‘14 bit CC - high first’ (a 14 bit CC where the high byte is sent first)
CC/note	Sets the MIDI CC or note number (0-127), or selects Channel Pressure.
Enabled?	Sets whether the mapping is enabled or disabled.
Symmetric?	Allows you to specify a mapping as symmetric. For a normal CC, the range 0-127 is mapped across the range set by the min & max settings below. For a symmetric CC, the value 64 is mapped to the midpoint of the range, and values above and below that are scaled by half the total range. A symmetric mapping is appropriate for a parameter which has a bipolar range around zero (for example, a pan position), where you want to be sure that a MIDI value of 64 gives you exactly zero in the middle.
Relative CC?	If no, the CC is a standard 0-127 value that directly sets the parameter. If yes, the CC is an up/down relative movement. Some MIDI controllers support this scheme as an alternative to soft takeover (see below).
Min	Sets the minimum parameter value that the mapping will set.
Max	Sets the maximum parameter value that the mapping will set.

I2C Mappings

This is where you set up mappings from I2C controllers.

Like MIDI mappings, I2C mappings directly set the algorithm parameters.

While in the I2C Mappings menu, pressing the right encoder enters “Learn” mode. While this is active, the module listens for I2C controllers and the first one received will be set as the controller for the current mapping. Press the right encoder again to leave Learn mode without altering the mapping. Note that there is a setting for whether setting up a mapping via Learn also disables any other mappings that were already using the controller.

The items that can be set up per mapping are as follows:

Parameter	Chooses the parameter to map.
Controller	Sets the I2C controller number.
Enabled?	Sets whether the mapping is enabled or disabled.

Symmetric?	Allows you to specify a mapping as symmetric. For a normal mapping, the range 0-16383 is mapped across the range set by the min & max settings below. For a symmetric mapping, the value 8192 is mapped to the midpoint of the range, and values above and below that are scaled by half the total range. A symmetric mapping is appropriate for a parameter which has a bipolar range around zero (for example, a pan position), where you want to be sure that a controller value of 8192 gives you exactly zero in the middle.
Min	Sets the minimum parameter value that the mapping will set.
Max	Sets the maximum parameter value that the mapping will set.

Performance page

This is where you assign an index within the performance page to a parameter. See the section on the performance page above.

Settings menu

The Settings menu lets you change various global options. Settings are stored internally, in flash memory, not on the MicroSD card.

Display

This submenu contains options for the module's screen.

Setting	**Description**
Contrast	Sets the display contrast (brightness).
Master contrast	Sets the display contrast (brightness) in coarse steps.
Flip	Allows you to flip the display and so use the module upside down.
Screensaver (minutes)	Sets the number of minutes after which the screensaver will kick in.
Screensaver style	Sets whether the screensaver is an animated graphic, a blank screen, or actually powers off the screen.
Wake on MIDI/I2C	If set, the display is woken from the screensaver when a mapped MIDI or I2C message is received.
Enter screensaver now	Immediately activates the screensaver.
Expert settings	This submenu lets you mess with very low level settings on the display driver chip. Most users will not want to change these. Just in case you do, the default values are given in parentheses in each menu name.

Support USB Video Class	Sets whether the module advertises USB Video Class support in its USB descriptor. Some USB MIDI hosts are known to fail if the descriptor includes this, so the option is provided to turn it off.

UI (User Interface)

This submenu contains options for how you interact with the module.

Setting	**Description**
Show button help	Sets the number of seconds to show the pop-up help. If set to 1, the help is permanently visible. If set to 0, the help is never shown automatically (but can still be shown via button 3).
Button 4 function	Chooses the menu to which button 4 is a shortcut when in the single algorithm view.
Button ordering	Chooses whether buttons 1-4 are numbered from left to right or right to left.
Primary encoder	Chooses which encoder is used to navigate the menus.
Pot threshold	Sets a threshold for movement of the pots.
CPU load display	Sets the update rate of the CPU load figures on the overview page. The options are “Responsive” and “Calm”.
Add algorithms bypassed	If set, new algorithms are added to the preset with their Bypass parameter enabled.
Performance page	Sets the layout of the performance page custom UI. See above.

Engine

Setting	**Description**
Sample rate	Allows you to choose the module’s sample rate (32, 44.1, 48, 88.2, or 96kHz). Does not take effect until you click on ‘> Apply <’.
Block size	Allows you to choose the module’s audio buffer size. Does not take effect until you click on ‘> Apply <’.
Allow USB rate change	If on, the USB host (e.g. your DAW) can change the module sample rate. If off, USB audio is fixed at the sample rate you choose. This setting takes effect after a reboot (since it changes the USB descriptors).

Globals

Setting	**Description**

Tuning A=	Sets a global tuning standard, the default being A=440Hz. The value set here corresponds to MIDI note 69 (A4).

Startup

Options for things that happen when the module powers up.

Setting	**Description**
Load last preset	Sets whether the module loads the most recently used (that is, saved or loaded) preset at startup. The “Reset preset” menu also has the effect of having the module forget the most recently used preset so it won’t be loaded at startup. Please note that the module does not “auto-save”. You always have to save manually, if you want to save any changes to the loaded preset.

Select Bus

Setting	**Description**
Speed multiplier	An experimental feature that allows you to run the Select Bus at higher than usual speeds. Leave this at ‘1’ (normal speed) unless you know you’re working with other Select Bus devices that also allow you to change the speed.

MIDI

Setting	**Description**
CC changes view	If set to ‘Parameter’, mapped MIDI CCs that change parameter values will also change the currently visible parameter (if the algorithm involved is the currently visible one). If set to ‘Parameter and algorithm’, the current algorithm will also be changed.
SysEx changes view	If enabled, MIDI SysEx messages that change parameter values will also change the currently visible parameter (if the algorithm involved is the currently visible one).
Soft takeover	Chooses the style of soft takeover applied to CCs. See below.
Send CCs	When to send MIDI CCs when parameters change. The options are 'Off', 'On preset load', 'On parameter change', or 'Both'.
Send CCs if changed by MIDI map	Sets whether CCs are sent when a parameter changes, if that parameter was changed by a MIDI mapping. This can help avoid loops, if for example the MIDI controller sending the CCs also sends received CCs back out.
Learn is exclusive	If enabled, when a MIDI CC mapping is set via 'Learn', any conflicting mapping of the same CC is unset.
SysEx Device ID	The SysEx device ID to respond to.

Program Change channel	The MIDI channel on which to respond to Program Change messages (see below).
Program Change uses Bank	Sets whether Bank Select (CC #0) is also used when responding to Program Change messages.
Program Change shows message	Sets whether a ‘preset loaded’ message is displayed when a preset is loaded by a MIDI Program Change message.
Forwarding	Sets whether incoming MIDI on the various ports (USB, Breakout, Select Bus) is passed on to the other ports. The option for “Breakout to Breakout” means that MIDI arriving on the breakout MIDI in will be passed to the breakout MIDI out.

MIDI CC soft takeover

If enabled, the disting NT supports the familiar 'soft takeover' paradigm for MIDI CCs, to avoid parameter jumps if the controller's physical position doesn't match that of the preset.

Three variants of soft takeover are available:

- **Sudden** – after loading a preset the controlling MIDI CC has to pass through the value that would set the current parameter value before assuming control. Before that happens, the CC has no effect.
- **Gradual** – similar to the way the module knobs work, the CC gradually takes control of the parameter value until the positions match, at which point soft takeover deactivates and the CC controls the parameter directly.
- **Always on** – like ‘Gradual’ except soft takeover does not deactivate; it is always applied for CC changes, just like it is for front panel knob changes.

Note that the disting NT also supports ‘relative’ MIDI CCs, which do away with the need for soft takeover. If a mapping is set to use relative CCs (see above), the soft takeover setting has no effect on it.

MIDI Program Change

If enabled, the module responds to MIDI Program Change messages by loading a preset. The following rules are followed to determine which preset is loaded:

- If a preset filename contains the program number preceded by a hash (‘#’), that preset is loaded. For example, if a Program Change is sent with a program number of 42, that will load a preset with “#42” anywhere in its name.
- Otherwise, the filenames are simply sorted alphabetically and the program number is used to index into the list.

I2C

Setting	Description
Address (as follower)	The module's I2C address.
CC changes view	If set to ‘Parameter’, mapped controllers that change parameter values will also change the currently visible parameter (if the algorithm involved is the currently visible one). If set to ‘Parameter and algorithm’, the current algorithm will also be changed.

Real-time clock

This submenu allows you to set the real-time clock, which is used for timestamping files when writing to the MicroSD card. Set the year, month, day, hour, and minute as desired and then select “> Set <” to actually set the time.

The time will be accurately updated while the module is powered, but will be lost when the module is powered off, since it lacks a battery. However, the MicroSD card is scanned at startup and the clock set to a time after the latest file found, so the times on any newly saved files will at least be later than those that came before.

Calibration

This menu contains various functions related to the calibration of the module. The module requires accurate calibration so that it can receive and generate accurate voltages and translate them to/from software.

The module is calibrated at the factory, so you shouldn’t need to worry about doing it yourself. If you do decide to calibrate the module yourself, you will need an accurate source of a +3V reference voltage. Other modules which are designed to deliver accurate pitch CVs are usually a good choice for this, for example, MIDI/CV converters.

Any changes you make to the calibration are not saved to flash unless you specifically do so with the “Save to calibration to flash” menu.

Zero input offsets

This function is not really calibration at all, but is a quick way to remove any DC offsets on the module inputs. Disconnect any patch cables from the module first.

Calibrate an input

This menu allows you to calibrate a single input. Choose an input, select “> Calibrate <”, and follow the displayed instructions.

Calibrate an output

This menu allows you to calibrate a single output. Note that it relies on Input 1 already being well

calibrated. Choose an output, select “> Calibrate <”, and follow the displayed instructions.

Save to calibration to flash

Saves any changes you’ve made to the calibration to flash memory.

Reset calibration

Resets the calibration to a vaguely sensible, but uncalibrated, state. Note that this function does not save anything to flash.

Read factory calibration

Loads the calibration as performed by the factory. Note that this function does not save anything to flash. Use “Save to calibration to flash” if you want to save the factory calibration as your new calibration.

View calibration

Allows you to see the various values that have been measured during calibration.

View input voltages

Allows you to see the voltages that the module thinks are on its various inputs. Ideally these would all be very close to zero when no patch cables are connected. The readings here will only be accurate if the module is well calibrated.

Audio recordings folder

This setting allows you to choose the folder for new recordings made by the Audio recorder algorithm.

Factory reset

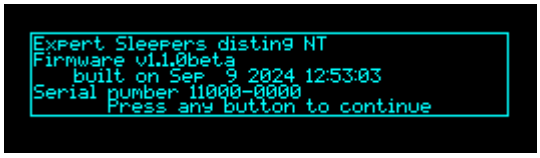
This menu restores all settings to their default values. Note that this does not affect the calibration.

Misc(ellaneous) menu

The ‘Misc’ menu contains a variety of functions that don’t fit anywhere else.

About

Displays the current firmware version information and the module serial number.



MicroSD card

This submenu contains some utilities related to the MicroSD card.

Remount

Lets the module know that you've removed the MicroSD card and reinserted it. Unlike a full-size SD card, a MicroSD card has no physical mechanism for letting the module know that the card was removed – as far as the module is concerned, the card simply stopped responding properly. Selecting this menu allows the module to reinitialise communication with the card.

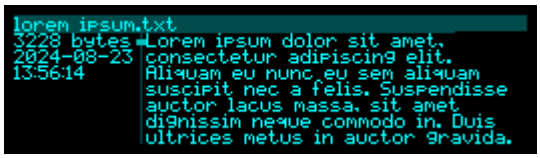
Test speed

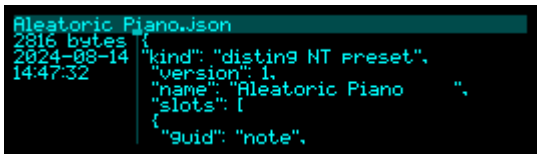
Performs a speed test on the card, and displays some statistics about the results.

Browse

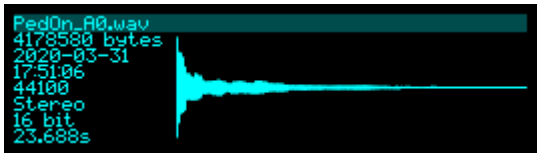
Allows you to browse the contents of the card, display information about files, and in some cases view the contents of the files.

Files with the extensions ".txt", ".json", and ".lua" can be viewed as text. Use the left pot or left encoder to scroll through the file.





Files with the extension ".wav" can be viewed as a waveform, with the file's sample rate, channel count, bit depth, and length (in seconds) displayed.



Enter USB disk mode

Video[39]

This reboots the module into a mode where the MicroSD card appears as a USB disk. All other functionality is suspended while in this mode.

If you connect the disting NT to a computer while in this mode it will appear as a removable drive, just as if you'd plugged in a USB thumb drive or portable harddrive. You can use this to manage files on the card without having to remove the card from the module and plug it into a MicroSD adaptor on your computer.

Remember to eject the drive on your computer before powering off the module or rebooting into

39 https://www.youtube.com/watch?v=nHvTzokMLYA

normal mode.

Press both encoders together to reboot the module and resume normal operation.

Reboot

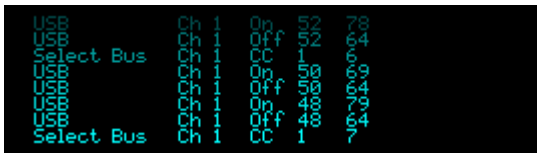
Reboots the module, as if you'd turned the power off and on again.

Enter bootloader mode

Puts the module into a mode in which it can communicate with the firmware update tool. Note that there is no way back from this mode other than to physically power cycle it.

MIDI monitor

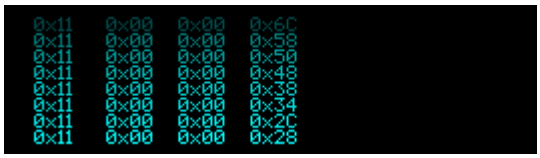
Runs a MIDI monitor, showing the MIDI messages that the module is receiving as text. Select the "> GO <" item to start the monitor.



The 'Show Clocks' option selects whether to include MIDI clock messages (F8h) in the display, which tend to flood out any useful information if shown (unless, of course, you're actually checking to see if you're receiving clocks or not).

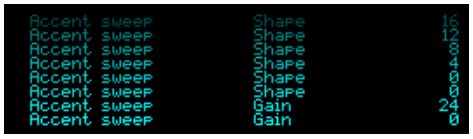
I2C monitor

Runs an I2C monitor, showing the I2C messages that the module is receiving as text.



Mappings monitor

Runs a mappings monitor, showing the parameters that have been changed as a result of MIDI or I2C mappings.



MIDI

This menu contains a couple of MIDI utilities.

- **All notes off** sends a MIDI “all notes off” message (CC 123, value 0) on all 16 MIDI channels, to all destinations (breakout, Select Bus, USB, and all algorithms).
- **All sound off** is similar but sends a MIDI “all sound off” message (CC 120, value 0).

Reset state

This instantly clears all audio delay buffers (echoes, reverbs etc.) in the preset. It could be useful when restarting a track (e.g. in rehearsal, or when doing overdubs), when you want to start from silence, without all the delay tails that you might have ringing on if you stop a track in the middle.

UI Scripts menu

This menu allows you to run a UI script. See below.

Algorithm-specific menu

The last menu, if present, offers functions specific to the current algorithm (the one highlighted in the algorithm overview). For example, the Granulator menu allows you to load and save samples. See each algorithm for details of its menu, if any.

Common algorithm features

A number of the disting NT algorithms share some features, which are described below.

Bypass

All algorithms have a ‘Bypass’ parameter. When enabled, the algorithm’s processing is completely skipped, and it consumes no CPU.



When bypassed, an algorithm appears in the overview ‘grayed out’.

A bypassed algorithm still responds to parameter automation via CV or MIDI/I2C (so you can enable/disable bypass remotely).

Note that this is not the same as what might be termed a “through” bypass. For example if you have a reverb taking input from input 1 and outputting to output 1, if you bypass it, you will have nothing on output 1 – not the dry signal from input 1.

Microtuning (Scala/MTS)

Microtuning is supported by both Scala files and MTS (MIDI Tuning Standard). Please see the section above for how files defining these tunings are arranged on the MicroSD card.

There are a number of parameters which together define how microtuning will be applied in the algorithm. The most important is ‘Microtuning’, which can be ‘None’, ‘Scala’, or ‘MTS’.

Scala

If ‘Microtuning’ is set to ‘Scala’, the tuning is defined by a Scala .scl file and optionally a .kbm file as well.

It is possible to send the files over MIDI SysEx, but note that sending Scala over MIDI is not a standard operation – to our knowledge it is currently only supported by our own tool, which you'll find in our GitHub[40]. Choose ‘From SysEx’ for the ‘Scala .scl’ or ‘Scala .kbm’ parameter to use the file sent in this manner.

It is much more common to use files directly, which in our case means from the MicroSD card.

40 https://github.com/expertsleepersltd/distingNT/tree/main/tools

Simply choose the files you want to use via the ‘Scala .scl’ and ‘Scala .kbm’ parameters.

The ‘Scala .kbm’ parameter has another option, which is ‘Automatic’. In this case, a .kbm file is not required, and the algorithm constructs the equivalent information from two further parameters, ‘Auto kbm root’ and ‘Auto kbm Hz’. Refer to the official .kbm format documentation here[41]. The automatic keyboard map generated is equivalent to the following .kbm file:

```
! Template for a keyboard mapping
!
! Size of map. The pattern repeats every so many keys:
<number of pitches in the .scl file>
! First MIDI note number to retune:
0
! Last MIDI note number to retune:
127
! Middle note where the first entry of the mapping is mapped to:
‘Auto kbm root’ value
! Reference note for which frequency is given:
‘Auto kbm root’ value
! Frequency to tune the above note to (floating point e.g. 440.0):
‘Auto kbm Hz’ value
! Scale degree to consider as formal octave (determines difference in pitch
! between adjacent mapping patterns):
<number of pitches in the .scl file>
! Mapping.
! The numbers represent scale degrees mapped to keys. The first entry is for
! the given middle note, the next for subsequent higher keys.
! For an unmapped key, put in an "x". At the end, unmapped keys may be left out.
0
1
2
<etc. up to the number of pitches in the .scl file minus 1>
```

MTS

If ‘Microtuning’ is set to ‘MTS’, the tuning can come either from MIDI SysEx messages, or loaded from the MicroSD card. This is selected by the ‘MTS .syx’ parameters. If this is set to ‘From SysEx’, the algorithm will use the tuning sent to it over MIDI; otherwise, it will load a SysEx dump from the card and use that.

The supported format is the bulk tuning dump as described here[42].

Chord/quantizer scales

The available scales are as follows.

Name	**Notes**	**Example (on C)**
Chromatic	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12	
Major	1, 3, 5, 6, 8, 10, 12	C D E F G A B
Natural Minor	1, 3, 4, 6, 8, 9, 11	C D E♭ F G A♭ B♭

41 https://www.huygens-fokker.org/scala/help.htm#mappings

42 https://midi.org/midi-tuning-updated-specification

Harmonic Minor	1, 3, 4, 6, 8, 9, 12	C D E ♭ F G A ♭ B
Dominant	1, 3, 5, 6, 8, 10, 11	C D E F G A B ♭
Fully Diminished	1, 3, 4, 6, 7, 9, 10, 12	C D E ♭ F F♯ A ♭ A B
Dominant Dim	1, 2, 4, 5, 7, 8, 10, 11	C D ♭ E ♭ E F♯ G A B ♭
Augmented	1, 4, 5, 8, 9, 12	C E ♭ E G A ♭ B
Whole Tone	1, 3, 5, 7, 9, 11	C D E F♯ G♯ A♯

Common polysynth features

A number of the disting NT algorithms are “polysynths”, and share some features, which are described below.

Min/max note

All polysynths have two parameters which define the range of MIDI note numbers that will be recognised - ‘Min note’ and ‘Max note’. These default to 0 and 127 respectively (i.e. the full MIDI range). Notes outside of this range will be ignored. Using these, you can set up keyboard splits.

These parameters also apply when using I2C and CV/gate, not just MIDI.

CV/gate setup

Polysynths can use up to 6 input busses as gates. Each gate can use up to 11 input busses as CVs. The CV busses follow immediately after the gate bus e.g. if you select input 1 as the gate, and choose to use two CVs for that gate, the CVs will be inputs 2 and 3.

While it might be convenient to have free assignment of each CV bus, this rapidly becomes unwieldy when you have a lot of busses to assign, so the present system is offered as a compromise between usability and flexibility.

Q: I thought CV/gates came in pairs? What does it mean to have one gate and multiple CVs?

A: It means that a gate will trigger a chord – one note per CV.

Gate velocity

Polysynth gates are “velocity sensitive” - the voltage of the gate is used as a velocity value, just like the velocity that you get from a MIDI keyboard. Typically this is used to scale note volume, but various synths might map it to filter cutoff, wavetable position, etc.

A gate voltage of 5V is “maximum velocity”.

Microtuning (Scala/MTS)

All polysynths support microtuning, as described above.

MPE

Polysynths support MPE (MIDI Polyphonic Expression). Per-note pitch bend, aftertouch, and the “Y dimension” (CC #74) are supported. Precisely what each polysynth does with this information is dependent on the synth.

MIDI channels for MPE are set using the ‘MIDI channel’ and ‘MPE channels’ parameters. The ‘MIDI channel’ sets the MPE common channel (typically channel 1). The ‘MPE channels’ parameter sets the range of channels that will be used for notes. If the common channel is 1, then the note channels will be from 2 up to a maximum. A special case is common channel as 16, in which case the note channels will go from 15 down to a minimum (this is an official MPE usage called the “upper zone”).

Chord generator/arpeggiator

The polysynths contain chord generators, which can add harmony notes to the notes you play, and an arpeggiator, to play those harmony notes as patterns instead of all at the same time.

The arpeggiator pattern is advanced by the gate, if playing via CV/gate, or by the MIDI or I2C note on message.

Harmony modes

There are two different ways in which the chord notes can be chosen.

The first, 'Shape’, builds up a chord using the original note as the root note, according to the shape and inversion parameters.

The second, 'SATB’, builds a chord according to common practice[43] four part harmony. (SATB stands for soprano/alto/tenor/bass.) The original note is assigned to one of the four voices according to the 'cantus firmus' parameter, and the remaining voices fill out the chord according to the root degree and position parameters. The root can also optionally be set from a CV input.

Note that the root is not the same as the key. In the key of C major for example, a root degree of II gives the chord D-F-A. In the key of D major, a root degree of I gives D-F♯-A.

Chords, arpeggios, and multiple CV inputs per gate

How these features interact warrants some clarification.

If there are multiple pitch CVs per gate, the chord generation is applied to each pitch CV. For example, if you supply the notes C and D, and set the chord generation to 'triad' in C major, you'll get the notes C, E, G, D, F, A.

If you enable arpeggiation, but not chords, then the arpeggiation is over the input pitch CVs. So for

43 https://en.wikipedia.org/wiki/Common_practice_period

example if you have three CVs and one gate, then the arpeggiator will run over the three notes supplied to the CV inputs. Note that this is useful even if you have a single pitch CV per gate, since you can still use the arpeggio range setting to get octaves.

If chords and arpeggiation are both enabled, then the arpeggio is over all the notes generated by the chords. In most cases, the notes are sorted and then the Up, Down, etc. direction imposed. The exception is 'As Played' mode – in this mode the notes are ordered according to the CV input that generated them. In the above example, 'As Played' mode would give you C, E, G, D, F, A, in that order, whereas 'Up' would give you C, D, E, F, G, A.

Chord/arp parameters

Name	Min	Max	Default	Unit	Description
Enable	0	2	0		Enables the chord generator and chooses which variant to use. The options are ‘Off’, ‘Shape’, and ‘SATB’.
Key	-12	12	0		Sets the key of the chord (C, D ♭ , D, etc.).
Mode	1	12	1		Sets the mode, that is, the rotation of the notes within the scale. If the selected scale is ‘Major’, the mode is displayed by one of the familiar names Ionian, Dorian, Phrygian, Lydian, Mixolydian, Aeolian, or Locrian.
Scale	0	8	1		Sets the scale to use to build the chord. See above.
Shape	0	13	0		In ‘Shape’ mode: sets the chord shape. See below.
Inversion	0	3	0		In ‘Shape’ mode: sets the chord inversion. For example the first inversion takes the first note of the chord and moves it an octave up, so the lowest note in the chord is now the second (e.g. C E G becomes E G C). See e.g. here[44] for a fuller explanation of inversions.
Root degree	0	7	1		In ‘SATB’ mode: the root degree. Specified manually (I-VII) or as ‘From CV’.
Root CV	0	64	0		The input bus to use as the root degree pitch if ‘Root degree’ is ‘From CV’.
Cantus firmus	0	3	3		In ‘SATB’ mode: which voice (soprano/alto/tenor/bass) corresponds to the original note.
Position	0	5	0		In SATB mode: chooses Close or Open position, or one of the T-voice options. See below.

44 https://en.wikipedia.org/wiki/Inversion_(music)#Inversions

Arpeggio	0	14	0		Enables the arpeggiator and chooses the arpeggiation mode. See below.
Range	1	3	1		When set to 1, the arpeggio is simply the notes formed by the chord. When set to 2 or 3, a copy of the chord is appended to the pattern, one or two octaves higher, creating a longer pattern that spans multiple octaves.
Arp reset	0	64	0		The input bus to use as the arpeggiator reset. A trigger pulse into this input will reset the arpeggiator back to step 1.
Break time	0	1000	0	ms	The 'break' time for chords. See below.
Break dir	0	2	0		The 'break' direction. See below.

Chord shapes

The available shapes are as follows.

Name	**Notes (within scale)**	**Example (in C major)**
None	1	C
Octave	1-1(8ve)	C C(8ve)
Two Octaves	1-1(8ve)-1(15ma)	C C(8ve) C(15ma)
Root/Fifth	1-5	C G
Root/Fifth + 8ve	1-5-1(8ve)	C G C(8ve)
Triad	1-3-5	C E G
Triad + 8ve	1-3-5-1(8ve)	C E G C(8ve)
Sus4	1-4-5	C F G
Sus4 + 8ve	1-4-5-1(8ve)	C F G C(8ve)
Sixth	1-3-5-6	C E G A
Sixth + 8ve	1-3-5-6-1(8ve)	C E G A C(8ve)
Seventh	1-3-5-7	C E G B
Seventh + 8ve	1-3-5-7-1(8ve)	C E G B C(8ve)
Ninth	1-3-5-7-2(8ve)	C E G B D

SATB mode – positions

In SATB mode, the ‘Position’ parameter controls the spacing between the various voices.

When set to ‘Close’, the voices adopt the closest triad notes possible to the cantus firmus. For example, if the cantus firmus is the soprano, the key is ‘C’, and the soprano voice is C4, then the alto, tenor, and bass notes will be G3, E3, and C3 respectively.

When set to ‘Open’, as compared to ‘Close’, the alto takes the tenor note, and the tenor takes the alto note dropped by an octave. The bass also drops an octave if required to keep it below the tenor. So in our example above, the alto, tenor, and bass notes will be E3, G2, and C2 respectively.

The remaining options for position are based on the ‘Tintinnabuli’[45] style developed by the composer Arvo Pärt. They make most sense when the cantus firmus is the alto part, which we will assume here – if not, the voices are calculated as explained and then simply permuted so what we call the alto here is the desired voice.

The bass voice is simply chosen as one octave below the alto. The tenor and soprano are chosen as follows:

T-vce +1 -1	Soprano is first position superior (the closest triad note above the alto). Tenor is first position inferior (the closest triad note below the alto).
T-vce +2 -1	Soprano is second position superior (the triad note one up from first position). Tenor is first position inferior.
T-vce +1 -2	Soprano is first position superior. Tenor is second position inferior (the triad note one down from first position).
T-vce +2 -2	Soprano is second position superior. Tenor is second position inferior.

Arpeggio modes

The options are as follows:

Name	Behaviour	Example (on C major triad)
Up	Notes are played from lowest to highest.	C E G C E G ...
Down	Notes are played from highest to lowest.	G E C G E C ...
Alt	Notes are played alternately up and down.	C E G E C E G ...
Alt2	Notes are played alternately up and down, repeating the top & bottom notes.	C E G G E C C E G ...
Up -8ve	See below.	
Down -8ve	See below.	

45 See e.g. https://en.wikipedia.org/wiki/Tintinnabuli

Alt -8ve	See below.	
Alt2 -8ve	See below.	
Step Up	The lowest note alternates with the other notes in the chord, in rising order.	C E C G C E C G ...
Step Down	The highest note alternates with the other notes in the chord, in descending order.	G E G C G E G C ...
Random	Notes are played in a random order.	
Random2	Notes are played in a random order, except that the same note cannot be played twice in a row.	
Random3	The notes in the chord are played in a random permutation, then another random permutation, and so on.	
As Played	Notes are played according to the order of their CV inputs.	

The “-8ve” modes differ from the basic modes in how they treat the Range parameter (above), for shapes which end in “+8ve”. As an example, consider the Triad+8ve shape in C major, which contains the notes:

C E G C(8ve)

If Range is set to 2, this pattern is repeated an octave higher, so modes Up/Down/Alt/Alt2 will arpeggiate the notes:

C E G C(8ve) C(8ve) E(8ve) G(8ve) C(15ma)

Note how C(8ve) is repeated. The “-8ve” modes skip this repeated note, so for example the Up-8ve mode will play:

C E G C(8ve) E(8ve) G(8ve) C(15ma) C E G …

Broken chords

The 'Break time' and 'Break direction' parameters allow you to automatically 'break' chords i.e. the notes are played one at a time, instead of simultaneously. With a small time, this gives a 'strumming' effect, though you can specify much longer times if you wish so each note is quite distinctly separated.

Note that this applies to multiple notes triggered when using a single gate input with multiple CV inputs, as well as when chord mode is activated.

The 'Break direction' allows you to specify whether the chord is played from bottom to top (‘Up’), from top to bottom (‘Down’), or alternately up and down (‘Alternating’).

The arpeggio reset input, if used, also serves to reset the alternating break direction to up.

Sustain mode

The polysynth ‘Sustain mode’ parameter sets the behaviour of the sustain function, when triggered either by MIDI or by the ‘Sustain’ parameter. The options are “Synth” (sustained notes cannot be retriggered) and “Piano” (sustained notes can be retriggered).

Plug-ins

As well as the various options for scripting (e.g. Lua Script algorithm, below), the disting NT supports *bona fide* plug-ins via a C++ API.

Here we'll focus on installing and load plug-ins from a user perspective, not on developing them.

Developers interested in the API are referred to our GitHub[46] and to our Discord server.

Currently plug-ins can define new algorithms. It is possible that in the future plug-ins might offer other new functionality. We're open to suggestions.

Plug-in gallery

There is a third party website called NT Gallery[47] which offers a browsable and searchable database of plug-ins the for the disting NT.

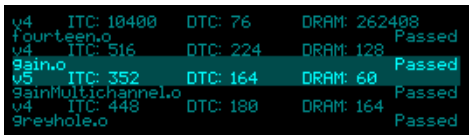
Installing plug-ins

Plug-ins are placed on the MicroSD card in the folder `/programs/plug-ins`

The expected file extension for plug-ins is `.o`

You can install plug-ins manually by simply copying the file to the MicroSD card. You can also install them via the NT Helper[48] app for macOS/Windows/Linux/iOS/Android.

The module scans the folder for plug-ins at startup (and if the card is ever remounted). You can view a list of what it found via the Misc / Plug-ins / View info… menu, which shows a screen like this:



The entry for each plug-in shows the filename (top left), whether it passed or failed scan (top right), and statistics for how much of various types of memory will be consumed if the plug-in is loaded. The API version that the plug-in was built with is shown below the filename.

Loading plug-in algorithms

Algorithms loaded from plug-ins are handled by the system just like any other algorithm. The only difference is that they need to be loaded before use. This can be done manually from the 'Add algorithm' menu; the module also attempts to load plug-ins automatically when loading a preset file that uses them.

46 https://github.com/expertsleepersltd/distingNT_API

47 https://nt-gallery.nosuch.dev

48 https://nosuch.dev/nt-helper/

Plug-in algorithms appear after the built-in algorithms in the list:



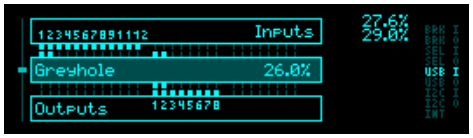
If the plug-in is not loaded, where you would normally see “> Add <” you’ll see “> Load <”:



If you click the encoder as usual, the plug-in will be loaded. If this succeeds, “> Load <” will change to the usual “> Add <”:



You can then add the algorithm as normal.



There is currently no way to unload plug-ins via the user interface, other than rebooting the module. There is a mechanism for resetting and rescanning the MicroSD card for plug-ins, which is intended as part of the development workflow.

Faust

One of the motivations for the disting NT’s C++ API was to be able to support Faust[49].

From their website:

> Faust (Functional Audio Stream) is a functional programming language for sound synthesis and audio processing with a strong focus on the design of synthesizers, musical instruments, audio effects, etc. created at the GRAME-CNCM Research Department.
>
> Faust targets high-performance signal processing applications and audio plug-ins for a variety of platforms and standards.
>
> The core component of Faust is its compiler. It allows to "translate" any Faust digital signal processing (DSP) specification to a wide range of non-domain specific languages such as C++, C, LLVM bit code, WebAssembly, Rust, etc. In this regard, Faust can be seen as an alternative to C++ but is much simpler and intuitive to learn.

49 https://faust.grame.fr

> Thanks to a wrapping system called "architectures," codes generated by Faust can be easily compiled into a wide variety of objects ranging from audio plug-ins to standalone applications or smartphone and web apps, etc.

We have created such an “architecture” to allow Faust programs to be compiled for the disting NT. This is currently in our GitHub here[50], though the intention is to contribute this to the actual Faust project so it will ultimately be part of the Faust installation itself.

Whether Faust is in fact more “intuitive” than C++ is open for debate, but there are undeniably some very attractive features:

- There’s a large library[51] of existing work written in Faust which can simply be picked up and compiled for any platform, now including the disting NT.
- There’s an online IDE[52], where you can interactively develop your code and run it, processing audio files that you supply or live audio from your computer.
- There’s even a GUI app[53] where you can construct your DSP code by dragging in blocks of processing and connecting them up with virtual wires.

It’s early days for Faust on the disting NT – if you have any experience to share, or thoughts on future development, please let us know. Currently it works quite well for audio effects (reverbs, filters etc.) and you’ll find working examples of these in our GitHub.

50 https://github.com/expertsleepersltd/distingNT_API/tree/main/faust

51 https://faustlibraries.grame.fr/libs/

52 https://faustide.grame.fr/

53 https://faustplayground.grame.fr

UI Scripts

The disting NT offers the ability to completely redefine the module's UI using the popular scripting language Lua[135].

It is anticipated that this will be particularly useful in a live performance scenario, where you might want to reduce the available controls to a few key items, and not want to see (or risk changing) the bulk of the preset.

Scripts can be loaded from anywhere on the MicroSD card, but the examples provided are in the 'ui_scripts' folder.

> This feature can probably best be considered a "technology preview" - it works, but there's so much more that could be added. We're very interested in your feedback on whether this is a useful feature to develop further.

Running a script

From the 'UI Scripts' menu, choose 'Run UI script', which will let you browse the MicroSD card for your chosen script.

The script completely takes over the module UI until it is exited. The combination of pressing all four pushbuttons at once is reserved as a means of exiting a UI script. You are also free to define your own choice of means of leaving the scripted UI via the script itself.

If the script fails for any reason the module will do its best to return any error information generated by the Lua interpreter.

Writing scripts

Further documentation of UI scripts and of Lua scripting on the disting NT in general can be found in the separate Lua scripting document, which can be found on the firmware download page (here[136]), right next to the download link for this manual.

135 https://www.lua.org

136 https://expert-sleepers.co.uk/distingNTfirmwareupdates.html

MIDI SysEx reference

The disting NT supports a variety of functions via MIDI System Exclusive (SysEx) messages.

The browser-based tools here[137] can be considered as informal documentation of these messages, as in most cases the messages only exist to support these tools.

SysEx Header

All SysEx messages are prefixed with a manufacturer's ID, which is a unique series of hex bytes assigned by the MIDI Manufacturers Association. The Expert Sleepers ID is 00H 21H 27H, so all SysEx messages relating to Expert Sleepers hardware will begin

F0 00 21 27

Messages for the disting NT follow this with 6DH:

F0 00 21 27 6D

followed by the module's SysEx ID (see the Settings, above)

F0 00 21 27 6D <SysEx ID>

and then with a byte to identify the specific type of message e.g.

F0 00 21 27 6D <SysEx ID> 01

16 bit values

Where '16 bit value' is indicated below, this is a sequence of 3 bytes:

<most significant 2 bits> <middle 7 bits> <least significant 7 bits>

Received SysEx messages

01H – Take screenshot

F0 00 21 27 6D <SysEx ID> 01 F7

This causes the disting NT to respond with a SysEx message containing a screenshot of what is currently on the module's display, using the '33H – Screenshot' format, below.

04H – Set real-time clock

F0 00 21 27 6D <SysEx ID> 04 <time MSB> <time> <time> <time> <time LSB> F7

This sets the module’s real-time clock to the time represented as a 32 bit number of seconds since the start of the epoch (January 1st, 1970).

137 https://github.com/expertsleepersltd/distingNT/tree/main/tools

07H – Wake from screensaver

F0 00 21 27 6D <SysEx ID> 07 F7

Resets the timer that controls the screensaver, and if the screensaver is active, dismisses it.

08H – Execute Lua

F0 00 21 27 6D <SysEx ID> 08 <text> F7

Executes the enclosed text as a Lua chunk. If there is any output, it is returned using the ‘09H – Lua output’ message.

09H – Install Lua

F0 00 21 27 6D <SysEx ID> 08 <algorithm index> <text> F7

Executes the enclosed text as a Lua chunk and installs the result as the program of the indexed Lua Script algorithm. If there is any output, it is returned using the ‘09H – Lua output’ message.

This message is intended to facilitate rapid iteration of Lua scripts during development. The final script still needs to be installed to the MicroSD card as usual.

11H – .scl file

F0 00 21 27 6D <SysEx ID> 11 <ignored byte> <file contents> F7

Sends a Scala .scl file for the module to use.

12H – .kbm file

F0 00 21 27 6D <SysEx ID> 11 <ignored byte> <file contents> F7

Sends a Scala .kbm file for the module to use.

20H – Set display mode

F0 00 21 27 6D <SysEx ID> 20 <mode> F7

Sets the module display mode.

22H – Request version string

F0 00 21 27 6D <SysEx ID> 22 F7

This causes the disting NT to respond with a SysEx message containing the module's version string as text, using the '32H – Message' format, below.

30H – Request number of algorithms

F0 00 21 27 6D <SysEx ID> 30 F7

Queries the number of algorithms – that is, the number of different algorithms that the module can

run, not the number of algorithms instantiated in the current preset. The response uses the '30H – Number of algorithms' format, below.

31H – Request algorithm info

F0 00 21 27 6D <SysEx ID> 31 <16 bit index> F7

Queries for details of the indexed algorithm (in the list returned by the 30H message). The response uses the '31H – Algorithm info' format, below.

32H – Add algorithm

F0 00 21 27 6D <SysEx ID> 32 <guid> <16 bit specification value> <16 bit specification value> <16 bit specification value> F7

Adds an algorithm to the current preset. If the requested algorithm is a plug-in, load the plug-in first.

33H – Remove algorithm

F0 00 21 27 6D <SysEx ID> 33 <algorithm index> F7

Removes an algorithm from the current preset.

34H – Load preset

F0 00 21 27 6D <SysEx ID> 34 <append> <NULL terminated ASCII string> F7

Loads or appends the preset specified by the filename string.

35H – New preset

F0 00 21 27 6D <SysEx ID> 35 F7

Starts a new preset (exactly as the "New Preset" menu).

36H – Save preset

F0 00 21 27 6D <SysEx ID> 36 <option> F7

Saves the preset. The option value can be 0 (prompt for file overwrite), 1 (never overwrite), or 2 (always overwrite).

37H – Move algorithm

F0 00 21 27 6D <SysEx ID> 37 <algorithm index> <new index> F7

Requests that an algorithm be moved to a new position in the list.

38H – Load plug-in

F0 00 21 27 6D <SysEx ID> 38 <guid> F7

Loads the plug-in that defines an algorithm with the given guid.

40H – Request algorithm guid

F0 00 21 27 6D <SysEx ID> 40 <algorithm index> F7

Queries the indexed algorithm in the current preset. The response uses the '40H – Algorithm guid' format, below.

41H – Request preset name

F0 00 21 27 6D <SysEx ID> 41 F7

Requests the current preset name. Responds with '41H – Preset name', as below.

42H – Request number of parameters

F0 00 21 27 6D <SysEx ID> 42 <algorithm index> F7

Requests the number of parameters in the indexed algorithm. Responds with '42H – Number of parameters', as below.

43H – Request parameter info

F0 00 21 27 6D <SysEx ID> 43 <algorithm index> <16 bit parameter number> F7

Requests information for the given parameter in the indexed algorithm. Responds with '43H – Parameter info', as below.

44H – Request all parameter values

F0 00 21 27 6D <SysEx ID> 44 <algorithm index> F7

Requests the current values of all parameters in the indexed algorithm. Responds with '44H – All parameter values', as below.

45H – Request parameter value

F0 00 21 27 6D <SysEx ID> 45 <algorithm index> <16 bit parameter number> F7

Requests the value of the given parameter in the indexed algorithm. Responds with '45H – Parameter value', as below.

46H – Set parameter value

F0 00 21 27 6D <SysEx ID> 46 <algorithm index> <16 bit parameter number> <16 bit value> F7

Sets the value of the given parameter in the indexed algorithm.

47H – Set preset name

F0 00 21 27 6D <SysEx ID> 47 <NULL terminated ASCII string> F7

Sets the preset name.

48H – Request unit strings

F0 00 21 27 6D <SysEx ID> 48 F7

Requests string descriptions of the possible parameter units (Hz, ms etc.). Responds with '48H – Unit strings', as below.

49H – Request enum strings

F0 00 21 27 6D <SysEx ID> 49 <algorithm index> <16 bit parameter number> F7

Requests the value strings for an 'enum' type parameter. Responds with '49H – Enum strings', as below.

4AH – Set focus

F0 00 21 27 6D <SysEx ID> 4A <algorithm index> <16 bit parameter number> F7

Sets the given parameter to be the currently active one in the module's display.

4BH – Request mappings

F0 00 21 27 6D <SysEx ID> 4B <algorithm index> <16 bit parameter number> F7

Requests the mappings for a parameter. Responds with ‘4BH – Mapping’, as below.

4DH – Set mapping

F0 00 21 27 6D <SysEx ID> 4D <algorithm index> <16 bit parameter number> <packed mapping data> F7

Sets the CV mapping for the given parameter.

4EH – Set MIDI mapping

F0 00 21 27 6D <SysEx ID> 4E <algorithm index> <16 bit parameter number> <packed mapping data> F7

Sets the MIDI mapping for the given parameter.

4FH – Set I2C mapping

F0 00 21 27 6D <SysEx ID> 4F <algorithm index> <16 bit parameter number> <packed mapping data> F7

Sets the I2C mapping for the given parameter.

50H – Request parameter value string

F0 00 21 27 6D <SysEx ID> 50 <algorithm index> <16 bit parameter number> F7

Requests the value string for a parameter. Responds with '50H – Parameter value string', as below.

51H – Set algorithm name

F0 00 21 27 6D <SysEx ID> 51 <algorithm index> <string> F7

Sets the algorithm’s custom name (as shown on the overview screen).

52H – Request parameter pages

F0 00 21 27 6D <SysEx ID> 52 <algorithm index> F7

Requests the parameter pages for an algorithm. Responds with '52H – Parameter pages', as below.

53H – Set string parameter value

F0 00 21 27 6D <SysEx ID> 53 <algorithm index> <16 bit parameter number> <string> F7

Sets the value of a string-valued parameter.

56H – Request paths

F0 00 21 27 6D <SysEx ID> 56 F7

Requests the current preset paths. Responds with ‘56H – Preset paths’, as below.

60H – Request number of algorithms

F0 00 21 27 6D <SysEx ID> 60 F7

Requests the number of algorithms in the current preset. Responds with ‘60H – Number of algorithms’, as below.

61H – Request routing

F0 00 21 27 6D <SysEx ID> 61 <algorithm index> F7

Requests routing information for the given algorithm. Responds with ‘61H – Routing information’, as below.

62H – Request CPU usage

F0 00 21 27 6D <SysEx ID> 62 F7

Requests CPU usage information. Responds with ‘62H – CPU usage’, as below.

7FH – Reboot

F0 00 21 27 6D <SysEx ID> 7F F7

F0 00 21 27 6D <SysEx ID> 7F 7F F7

There are two forms of this message. The first simply reboots the module. The second (with the double 7FH) reboots the module into bootloader mode.

Sent SysEx messages

09H – Lua output

F0 00 21 27 6D <SysEx ID> 09 <text> F7

This message may be transmitted in response to ‘08H – Execute Lua’ or ‘09H – Install Lua’.

30H – Number of algorithms

F0 00 21 27 6D <SysEx ID> 30 <16 bit value> F7

This message is transmitted in response to ‘30H – Request number of algorithms’.

31H – Algorithm info

F0 00 21 27 6D <SysEx ID> 31 <16 bit index> <4 byte guid> <number of specifications> [<specification min> <max> <default> <type> …] <NULL terminated algorithm name> [<NULL terminated specification name> …] <plugin?> <loaded?> <NULL terminated filename> F7

This message is transmitted in response to ‘31H – Request algorithm info’.

32H – Message

F0 00 21 27 6D <SysEx ID> 32 <NULL terminated ASCII string> F7

This message is transmitted in response to any request for a string e.g the version string.

33H – Screenshot

F0 00 21 27 6D <SysEx ID> 33 00 <screenshot data> F7

This message is transmitted in response to a '01H – Take screenshot' message.

40H – Algorithm guid

F0 00 21 27 6D <SysEx ID> 40 <index> <4 byte guid> <string> F7

This message is transmitted in response to a ‘40H – Request algorithm guid’ message. The string is the algorithm’s custom name, as shown on the overview screen.

41H – Preset name

F0 00 21 27 6D <SysEx ID> 41 <NULL terminated ASCII string> F7

Contains the current preset name.

42H – Number of parameters

F0 00 21 27 6D <SysEx ID> 42 <algorithm index> <16 bit number of parameters> F7

Contains the number of parameters in the indexed algorithm.

43H – Parameter info

F0 00 21 27 6D <SysEx ID> 43 <algorithm index> <16 bit parameter number> <16 bit minimum> <16 bit maximum> <16 bit default> <unit> <ASCII string> <scaling | flags> F7

Contains information for the given parameter in the indexed algorithm. Scaling is a 2 bit value and uses the low 2 bits.

44H – All parameter values

F0 00 21 27 6D <SysEx ID> 44 <algorithm index> [<16 bit value>] F7

Contains the current values of all parameters in the indexed algorithm. The high byte of each 16 bit value uses the spare space to include flags in bits 2 and up.

45H – Parameter value

F0 00 21 27 6D <SysEx ID> 45 <algorithm index> <16 bit parameter number> <16 bit value> F7

Contains the value of the given parameter in the indexed algorithm. The high byte of the 16 bit value uses the spare space to include flags in bits 2 and up.

48H – Unit strings

F0 00 21 27 6D <SysEx ID> 48 <number of strings> [<ASCII string>] F7

Contains an array of string descriptions of the possible parameter units (Hz, ms etc.).

49H – Enum strings

F0 00 21 27 6D <SysEx ID> 49 <algorithm index> <16 bit parameter number> <number of strings> [<ASCII string>] F7

Contains the value strings for an 'enum' type parameter.

4BH – Mapping

F0 00 21 27 6D <SysEx ID> 4B <algorithm index> <16 bit parameter number> <version number> <mapping data> F7

Contains the mapping information for one parameter.

50H – Parameter value string

F0 00 21 27 6D <SysEx ID> 50 <algorithm index> <16 bit parameter number> <ASCII string> F7

Contains a value string for a parameter.

52H – Parameter pages

F0 00 21 27 6D <SysEx ID> 52 <algorithm index> <number of pages> [<title> <number of parameters> [<2 byte parameter index>]] F7

Contains the parameter pages for an algorithm.

56H – Preset paths

F0 00 21 27 6D <SysEx ID> 56 <preset path> <End-of-chain preset path> <Preset save folder> F7

Sent in response to ‘56H – Request paths’, as above. Each path is null-terminated, and may simply be a single null character if empty.

60H – Number of algorithms

F0 00 21 27 6D <SysEx ID> 60 <count> F7

Sent in response to ‘60H – Request number of algorithms’, as above.

61H – Routing information

F0 00 21 27 6D <SysEx ID> 61 <algorithm index> <routing data> F7

Sent in response to ‘61H – Request routing’, as above.

62H – CPU usage

F0 00 21 27 6D <SysEx ID> 62 <percentage bytes> F7

Sent in response to ‘62H – Request CPU usage’, as above. Each byte is a percentage (0-100). The first is the audio thread total usage; the second is the overall usage. Following these is one byte for each algorithm in the preset.

7AH – SD card access

F0 00 21 27 6D <SysEx ID> 7A <variable> F7

Message 7AH defines a whole sub-protocol for accessing the module’s MicroSD card. See below for details.

7FH – Reboot

F0 00 21 27 6D <SysEx ID> 7F F7

Reboots the module.

MicroSD card access

The module’s MicroSD card can be accessed via the 7AH message. The format of the message is

F0 00 21 27 6D <SysEx ID> 7A <command> [<command specific data>] <checksum> F7

The checksum is chosen so that the low 7 bits of the sum of the message bytes from the command up to and including the checksum are zero.

If a command fails, the module responds with

F0 00 21 27 6D <SysEx ID> 7A 01 <NULL terminated error message> F7

If a command succeeds, the module responds with

F0 00 21 27 6D <SysEx ID> 7A 00 <command> [<command specific data>] F7

The available commands are as follows.

01H – Directory listing

F0 00 21 27 6D <SysEx ID> 7A 01 <folder path string> <checksum> F7

The response data is a NULL terminated array of entries, one for each item in the directory, formatted as

40H | <FAT32 attrib byte>
<FAT32 date, short as 3 bytes>
<FAT32 time, short as 3 bytes>
<file size, 64 bit value as 10 bytes>
<NULL terminated filename>

02H – Download file

F0 00 21 27 6D <SysEx ID> 7A 02 <file path string> <checksum> F7

The response data is the entire file, with each byte in the file transmitted as two bytes, high nybble first.

03H – Delete file

F0 00 21 27 6D <SysEx ID> 7A 03 <file path string> <checksum> F7

Deletes the specified file. The response is simply success or failure.

04H – Write file

F0 00 21 27 6D <SysEx ID> 7A 04 <file path string> <create always?> <offset> <count> <file data> <checksum> F7

Writes data to the specified file. “Create always?” is a flag – if true, the file will always be created anew (and any existing file of the same name deleted), otherwise the data will be appended to any existing file. “Offset” and “Count” are 64 bit values represented as 10 bytes, high bits first. The file data is transmitted as two bytes per byte in the file, high nybble first.

The response is simply success or failure.

05H – Rename

F0 00 21 27 6D <SysEx ID> 7A 05 <file path string> <new path string> <checksum> F7

Renames or moves the specified file. The response is simply success or failure.

06H – Remount

F0 00 21 27 6D <SysEx ID> 7A 06 <checksum> F7

Remounts the MicroSD file system. The response is always success.

07H – New folder

F0 00 21 27 6D <SysEx ID> 7A 07 <folder path string> <checksum> F7

Creates a new folder. The response is simply success or failure.

08H – Rescan plug-ins

F0 00 21 27 6D <SysEx ID> 7A 08 <checksum> F7

Attempts to reset the plug-in list and rescan the MicroSD card for plug-ins. The response is simply success or failure. (Failure would indicate that one or more plug-ins is still in use.)

I2C reference

In general the disting NT acts as a follower on the I2C bus, not as a leader. It receives messages in the following format:

<address> <command> <optional bytes according to command>

A table of supported commands is below.

Some commands are “get” commands. The disting NT expects the get command to be followed immediately by a read of the requested data.

Devices on an I2C bus have an address, which a sending device uses to identify the intended recipient. The disting NT's address is set in the Settings (see above).

Value ranges

Preset, algorithm & parameter numbers are 1-based.

Voltages (and related quantities e.g. pitch) are signed and scaled as 16384 ↔ 10V.

Velocities are 0-16384.

I2C Channels

The note on/off messages have the concept of a ‘channel’, which is entirely analogous to a MIDI channel. Algorithms which receive notes will have a parameter to set which I2C channel to receive on, and will only respond to notes on that channel. Messages which don’t explicitly contain a channel use the last channel set via the 0x6B message.

The “current algorithm”

Much of this I2C protocol is based on that developed for the disting EX, which ran a single algorithm, so commands to set e.g. algorithm parameters didn’t need to specify which algorithm was the target.

Rather than develop a whole new protocol, the current solution (or workaround, depending on your point of view) is to define a “current algorithm”, which all algorithm-relative commands refer to. This is set using the 0x46 or 0x47 messages, using a parameter number of 255. The value set to parameter 255 (which is an invalid parameter number on all existing algorithms) will be used as the index (1-based) in the preset of the “current algorithm”.

Messages

Controllers/parameters

Set i2c controller X to value Y

<address> 0x11 <controller number> <value MSB> <value LSB>

These messages are used via the Mappings (see above) to control algorithm parameters.

Set parameter X to value Y (using the actual parameter value)

<address> 0x46 <parameter number> <value MSB> <value LSB>

As a temporary workaround, parameter number 255 is used to specify the target algorithm for subsequent parameter change messages.

Set parameter X to value Y (using 0..16384 range)

<address> 0x47 <parameter number> <value MSB> <value LSB>

The 0-16384 value range will be scaled to the actual parameter value range.

Get parameter value

<address> 0x48 <parameter number>

Returns 2 bytes.

Get parameter min

<address> 0x49 <parameter number>

Returns 2 bytes.

Get parameter max

<address> 0x4A <parameter number>

Returns 2 bytes.

Notes

Set pitch for note id.

<address> 0x54 <note id> <pitch MSB> <pitch LSB>

Note on for specified note id.

<address> 0x55 <note id> <velocity MSB> <velocity LSB>

Note off for specified note id.

<address> 0x56 <note id>

All notes off.

<address> 0x57

Set pitch for note id, with channel.

<address> 0x68 <channel> <note id> <pitch MSB> <pitch LSB>

Note on for specified note id, with channel.

<address> 0x69 <channel> <note id> <velocity MSB> <velocity LSB>

Note off for specified note id, with channel.

<address> 0x6A <channel> <note id>

Set channel for subsequent note based commands.

<address> 0x6B <channel>

MIDI

Send MIDI message from the breakout.

<address> 0x4F <status> <optional data byte 0> <optional data byte 1>

Send MIDI message from the Select Bus.

<address> 0x50 <status> <optional data byte 0> <optional data byte 1>

Send MIDI to the disting NT.

<address> 0x81 <byte 0>

<address> 0x82 <byte 0> <byte 1>

<address> 0x83 <byte 0> <byte 1> <byte 2>

Messages 0x81-83 allow you to send arbitrary MIDI to the disting NT, as if the I2C connection were just another MIDI cable into the module.

The various message forms are a convenience (e.g. the 3 byte form is convenient for note on/off or CC messages), but in terms of processing, the disting NT treats them all as a byte stream.

Updating the firmware

The disting NT's firmware is updated over its USB connection. You will need a computer and the appropriate USB cable to connect the module to it.

You can jump directly to any firmware version – you don't need to apply them incrementally. You can upgrade or downgrade at will, but note that older firmware versions may not be able to read presets saved with newer versions; similarly, if you downgrade you may find that your settings reset to defaults.

There are a number of ways of updating the firmware. The simplest for most users will be to use the NT Helper[138] app for macOS/Windows/Linux. Skip down to "Method 1: NT Helper" if you're most users.

We also support two somewhat lower-level methods of flashing the firmware – one using a GUI tool written by the chip vendor NXP, and one using a script. The latter will probably be quicker and simpler if you're comfortable with using the terminal/command prompt on your computer.

In both cases, you first need to:

- Download the firmware from our website here[139].
- Put the disting NT into bootloader mode, which you do via the 'Misc' menu. Select "Enter bootloader mode", click past the "Are you sure?", and the module will show you the message "Entering serial downloader" and appear to hang.

If you decide you don't actually want to flash new firmware at this point, simply turn the module off and on again.

Method 1: NT Helper

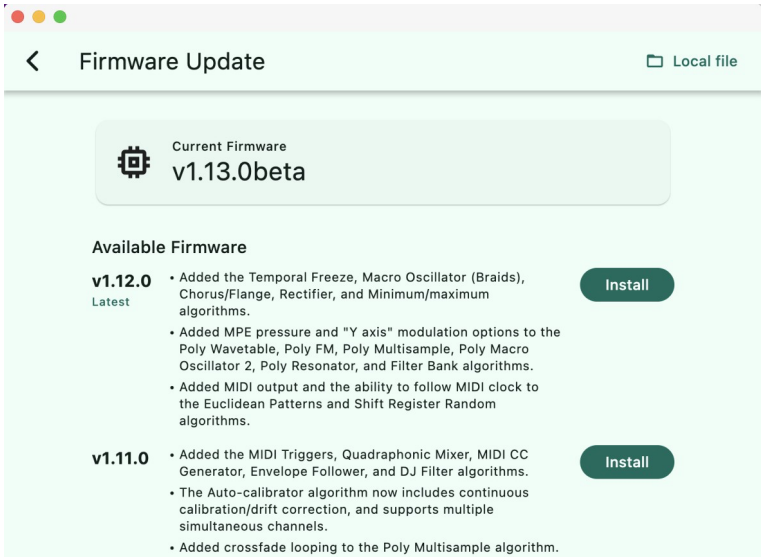
Instructions are on the author's page here[140]. You can access the firmware update page by clicking on the version number (bottom right) or via the menu (top right).

The app allows you to simply choose an official release directly from our website, or to install a file that you've downloaded locally.

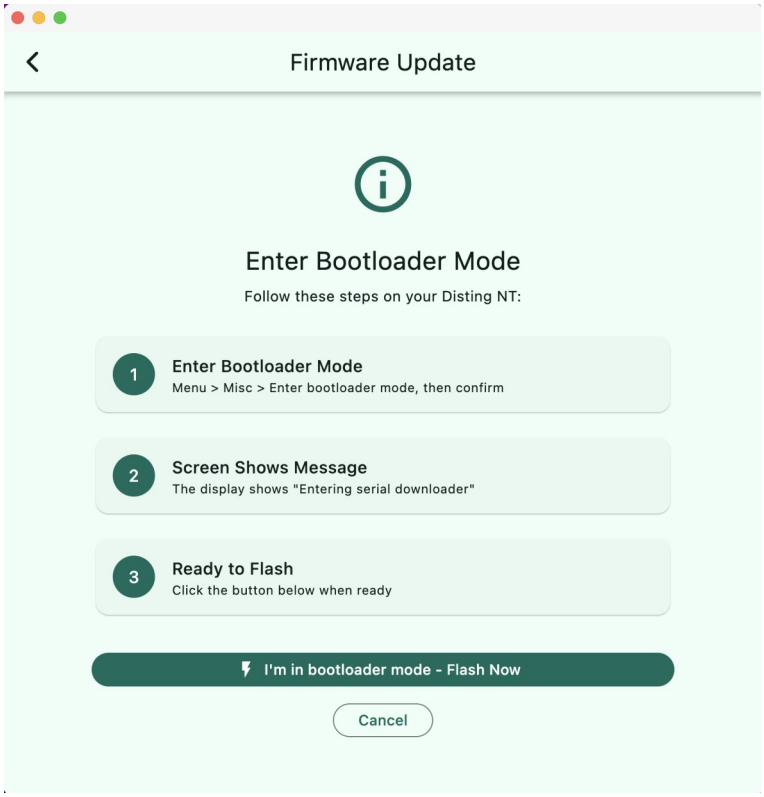
138 https://nosuch.dev/nt-helper/

139 https://expert-sleepers.co.uk/distingNTfirmwareupdates.html

140 https://nosuch.dev/nt-helper/#firmware-update



Once you've chosen the version to install, the app walks you through the update process.



Method 2: NXP GUI tool

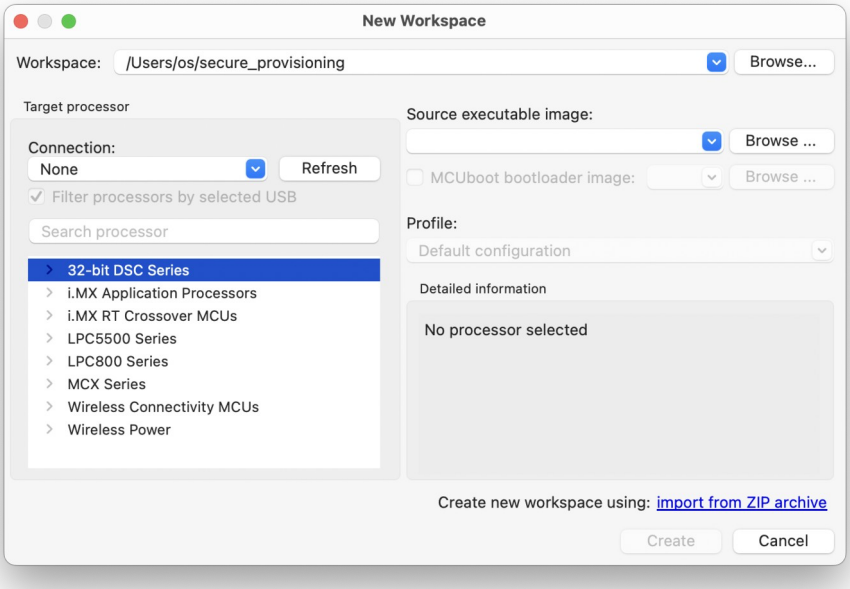
When downloading the firmware, do not let the browser automatically unzip the file (in Safari the option is "Open safe files after downloading" - that needs to be off). It is most unlikely to work if you attempt to make a zip from the unarchived files.

Download and install the correct version of the "MCUXpresso Secure Provisioning Tool". It is available for macOS, Linux, and Windows. Note that various versions of the tool may be available.

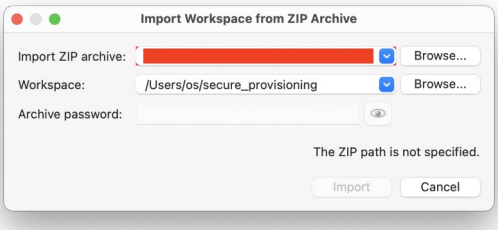
You need to use the correct version that matches the disting NT firmware package that you want to install. The tool version for each firmware version is noted on the firmware download page.

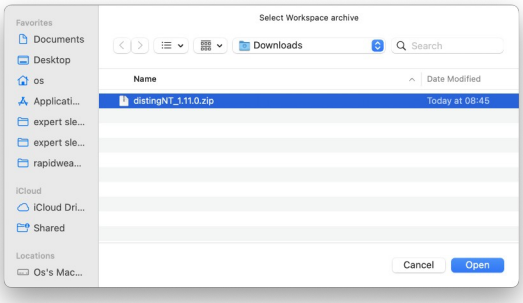
Download the tool for your platform of choice from this page[141]. The NXP website requires you to sign up for an account to do so and asks for a surprising amount of information. You can of course just make this up if you prefer to remain anonymous.

When you first run the tool, you should be presented with the “New Workspace” dialog:

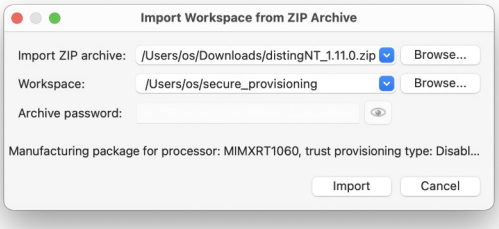


Click the text at the bottom that says “import from ZIP archive”. Browse for the firmware zip file that you downloaded.



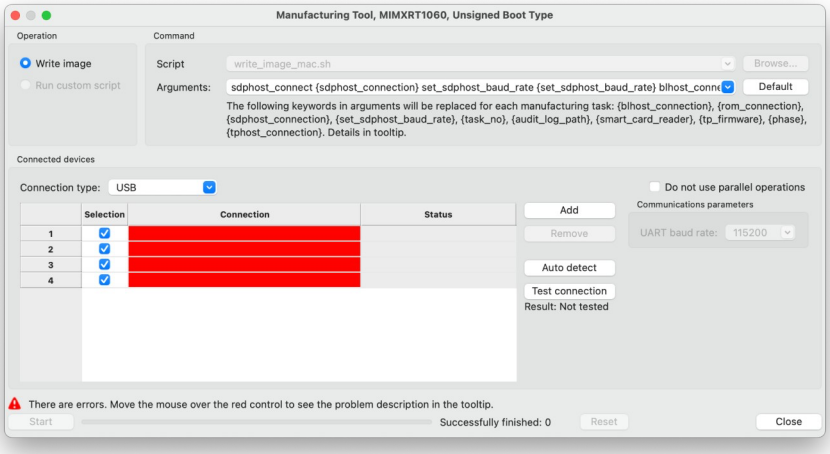


141 https://www.nxp.com/design/design-center/software/development-software/mcuxpresso-software-and-tools-/mcuxpresso-secure-provisioning-tool:MCUXPRESSO-SECURE-PROVISIONING

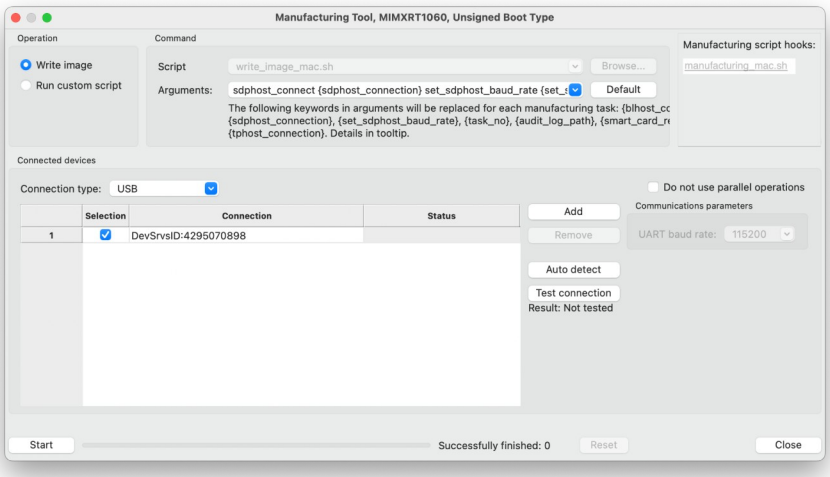


You can also choose where the tool creates its “workspace” - a folder containing a variety of temporary files that you can delete later, or just leave for next time.

Click the “Import” button.

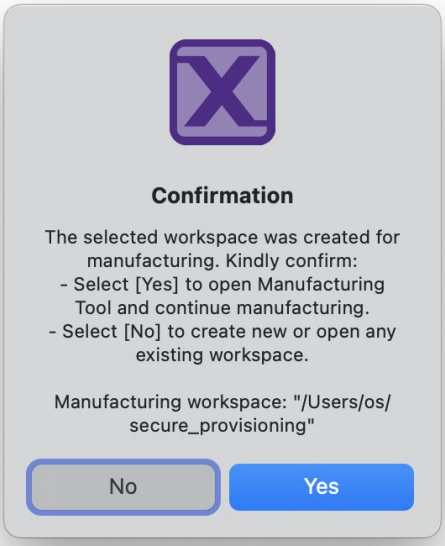


You should now see a window titled “Manufacturing Tool”. The connection section may be all red; if so, click “Auto detect”. If a disting NT in bootloader mode is connected via USB, the tool will now find it.



You can then click on the “Start” button (bottom left) to actually perform the firmware update. This should take maybe 15 seconds, and show “Success” when done. Turn the module off and back on to resume normal operation. You may like to check that the “About” screen on the disting NT shows the expected firmware version.

Next time you run the tool, you may see this dialog:



Click “Yes” if you want to reflash the same firmware as before (perhaps you’re the lucky owner of two disting NTs). Otherwise, click “No”, which will take you back to the “New Workspace” dialog as above.

Method 3: Script

As mentioned above, this method is provided for users who prefer a command line approach over using a GUI application. In preparation for this:

1. Install SPSDK. Instructions are here[142]. Note that you may also need to install development tools; for example on macOS you may need to install Xcode.
2. Download the scripts from our GitHub. You might choose to clone the project using git or GitHub Desktop, or simply download a release from the releases page[143]. The scripts are in the ‘flash’ folder.

To flash the firmware:

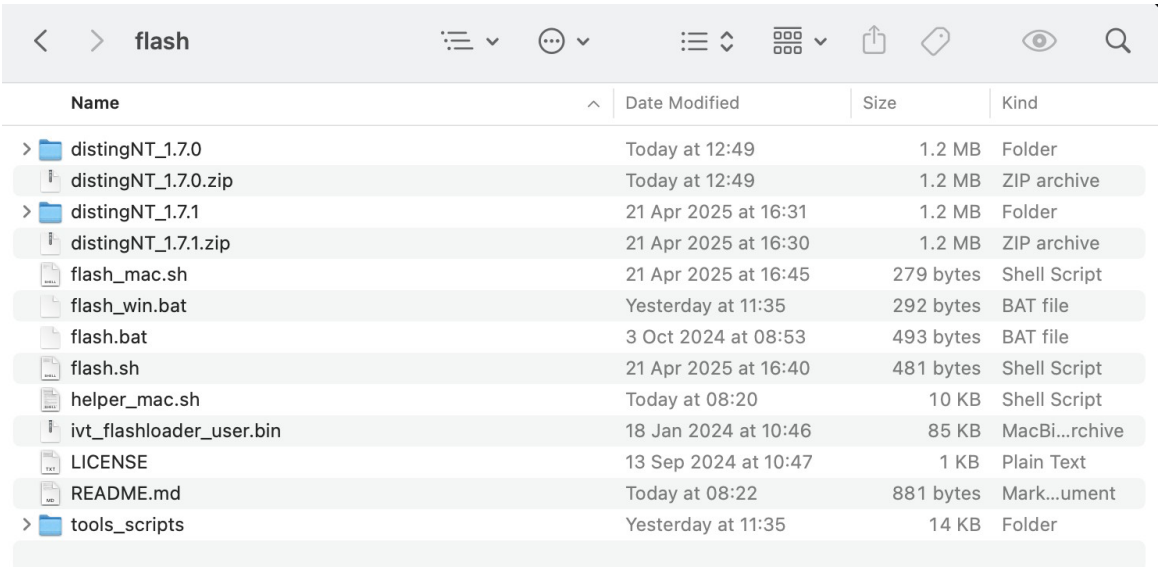
1. Put the downloaded firmware package into the ‘flash’ folder, and unzip it.
2. Put the disting NT into bootloader mode as described above.
3. Run the script ‘flash_mac.sh’ (macOS/Linux) or ‘flash_win.bat’ (Windows), supplying it the

142 https://spsdk.readthedocs.io/en/latest/examples/_knowledge_base/installation_guide.html

143 https://github.com/expertsleepersltd/distingNT/releases

folder name of the firmware you want to install.

For example, your ‘flash’ folder might look like this (here we’ve downloaded two firmware packages and unzipped them):



You would then run e.g.:

```
(base) oss-mac-studio:flash os$ ./flash_mac.sh distingNT_1.7.0
### Parse input arguments ###
### Check presence of FlashLoader ###
### FlashLoader is not running yet, download and run it ###
### Check communication with target bootloader ###
```

Acknowledgments

The Kirbinator algorithm was designed in collaboration with Simon Kirby[144].

Many thanks to all the beta testers.

Fonts

The small pixel font used throughout the UI is ‘PixelMix’ by Andrew Tyler, for which Expert Sleepers Ltd has a commercial license.

The large font is Microsoft Selawik (Copyright 2015, Microsoft Corporation), licensed under the SIL Open Font License version 1.1.

The tiny font used in parts of the UI is an adapted version of 'Tom Thumb' by Robey Pointer (MIT license).

Freetype

Portions of this software are copyright © 2023 The FreeType Project (www.freetype.org). All rights reserved.

Lua



Copyright © 1994–2024 Lua.org, PUC-Rio.

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

144 https://www.simonkirby.net